

OPIS PROJEKTOG ZADATKA

Organizacija plesnih natjecanja je dugotrajan i zahtjevan posao. Organizatori moraju upravljati velikim brojem prijava, kategorija, sudaca i rezultata, dok voditelji klubova i suci često nemaju jedinstvenu digitalnu platformu koja bi olakšala suradnju i komunikaciju. Svrha aplikacije DanceArena bi bila olakšati organizaciju i prijavu na turnire te njihovo ocjenjivanje na jednoj sveobuhvatnoj platformi.

Korisnici se na DanceArenu prijavljuju, odnosno registriraju u sustav koristeći svoju e-mail adresu i lozinku. Također, pri registraciji u sustav korisnik mora izabrati želi li koristiti taj račun kao organizator, voditelj kluba ili sudac. Voditelji klubova i suci mogu koristiti aplikaciju potpuno besplatno, dok organizatori moraju plaćati godišnju članarinu

Organizatori u aplikaciju otvaraju plesna natjecanja, pri čemu su dužni definirati datum i lokaciju održavanja, opis događaja i popis kategorija u kojima se grupe mogu natjecati te kotizaciju koja se plaća prilikom prijave nastupa. Kategorije natjecanja su dobne (djeca, juniori i seniori), različiti stilovi plesa (balet, hip hop, jazz, step, break, ...) i različite veličine grupa (solo, duo, mala grupa (3-8 plesača), formacija (>9)). Također, tijekom otvaranja natjecanja organizator mora prijaviti minimalno 3 i uvijek neparan broj suca. Sucima koji nisu registrirani u aplikaciji se šalje poziv za registraciju na mail koji organizator unese. Jednom kad je natjecanje otvoreno, organizatori mogu pregledavati prijavljene nastupe te potvrditi ili odbiti prijave. Kad organizator zaključi prijave, generira se pdf dokument u kojem je startna lista natjecanja grupirana po kategorijama te ga mogu preuzeti svi korisnici koji sudjeluju u natjecanju.

Voditelji klubova kreiraju profile s podacima o nazivu, lokaciji, kontaktu i e-mailu kluba te mogu prijaviti svoje grupe na otvorena natjecanja. Za svaki nastup potrebno je unijeti naziv koreografije, duljinu trajanja te ime koreografa. Također je potrebno priložiti muziku koja će se koristiti za nastup te odabrati kategoriju unutar koje će se natjecati (dob, stil, veličina grupe).

Suci mogu postavljati ocjene od 0 do 30 bodova za koreografije tijekom održavanja natjecanja za koja su prijavljeni. Po završetku natjecanja, računaju se ukupni bodovi za svaki nastup, te se za svaku kategoriju rangiraju nastupi od prvog mjesta nadalje. Rezultati natjecanja vidljiv je javno, dok ocjene mogu vidjeti samo voditelji klubova za svoje nastupe.

U sustavu postoje i administratori koji postavljaju cijenu članstva, upravljaju korisnicima i nadgledaju objave i prijave.

CILJ PROJEKTOG ZADATKA

Glavni cilj naše aplikacije je pružati platformu organizatorima, vlasnicima klubova i sucima koja bi im olakšala organizaciju i prijavljivanje na natjecanja. S jednom sveobuhvatnom platformom bi bilo jednostavnije promovirati i pronaći natjecanja te bi se smanjio rizik greške u komunikaciji ili manjka informacija jer bi sve specifikacije bile napisane i uvijek dostupne.

Aplikacija bi također olakšala ocjenjivanje sucima, kojima bi bila automatski prezentirana opcija za bodovanje koreografija natjecanja na koje su prijavljeni.

DanceArena bi ujedinila sve tehničke zahtjeve organizacije i izvođenja plesnih natjecanja na brz, efikasan i pristupačan način korisnicima, bez potrebe za kreiranjem osobnih web stranica ili sustava.

PROBLEMATIKA PROJEKTOG ZADATKA

Većina plesnih natjecanja se i dalje organiziraju ručno. Grupe se prijavljuju preko maila ili obrazaca, kotizacije se plaćaju izvan sustava a liste i rezultati su sklone greškama jer se slažu manualno. Makar trenutni sistem otvaranja i vođenja natjecanja radi, ima dosta prostora za poboljšanje.

Korištenjem DanceArene kao singularne platforme za sve navedene funkcije bi spriječilo razne greške koje se mogu provući pri ručnoj organizaciji. Problemi poput: nepotpunih ili pogrešnih prijava, manjak informacija o mjestu i vremenu turnira, greška u bodovanju nastupa, otežanog praćenja rezultata i mnogi drugi se ne bi javljali pri korištenju DanceArene.

S druge strane, korištenje digitalne platforme otvara nove potencijalne rizike na koje treba paziti, poput sigurnosti korisnika prilikom prijave u sustav i kartičnog plaćanja. Stoga koristimo OAuth2 prijavu, koja je jednostavna za koristiti i sigurna jer ne dijeli šifru direktno s aplikacijom, smanjujući rizik vanjskog pristupa korisničkim podacima. Rizik kartičnog plaćanja bismo riješili korištenjem sustava PayPal, koji garantira korisniku sigurnost pri plaćanju u aplikaciji.

POSTOJEĆA SLIČNA RJEŠENJA

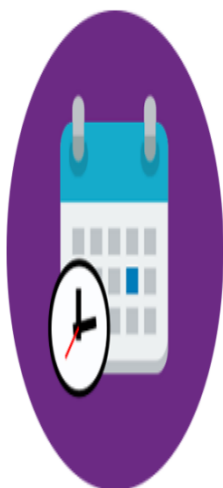
Na tržištu već postoji par sličnih aplikacija koje se bave olakšavanjem i automatizacijom plesnih natjecanja. Jedna od najkorištenijih takvih aplikacija je DanceBUG, koja također nudi olakšano otvaranje i prijavu

Welcome to DanceBUG®

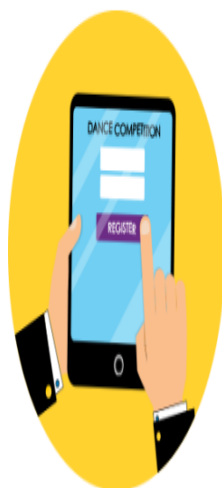
Whether you are a Parent, Dancer, Studio Owner, or a Competition Director, you will find what you need at DanceBUG.com.



PURCHASE & VIEW VIDEOS &
PHOTOS



SCHEDULE YOUR EVENT



REGISTER FOR COMPETITIONS



ACCESS VIDEO JUDGE

* SLIKA 1 : sažetak funkcija prikazanih na DanceBUG stranici

Nakon registracije, DanceBUG također ima različite vrste korisničkih računa. Organizatori imaju pristup administrativnom sučelju putem kojeg mogu kreirati nova natjecanja, upravljati prijavama, generirati rasporede nastupa i pratiti rezultate. Plesni klubovi mogu putem svojih profila prijavljivati grupe i solo nastupe na otvorena natjecanja te pratiti status i informacije tih prijava.



Theatre Arts

2020 performances

Theatre Arts is using DanceBUG.com for its 2020 season and you can purchase your recital video access now!



ONLY \$35.00!

You will get ONLINE access to ALL of your 2020 performances for the low price

WHAT'S INCLUDED!

2020 Theatre Arts Dance Recital

Sunday, July 26, 2020 at 2:00 PM & 6:00 PM
Union Performing Arts Center, Tulsa, OK

CONTINUE

- SLIKA 2 : prikaz sučelja za kupovinu medijskih sadržaja probe u DanceBUG aplikacije

Aplikacija DanceBUG također pruža neke funkcionalnosti koje nisu prisutne u DanceAreni, poput online kupnje videozapisa nastupa i proba s natjecanja. Sustav je integriran s online sustavima plaćanja te automatski obrađuje uplatu i dodjeljuje korisnicima pristup kupljenim snimkama. Uz to, DanceBUG ima mogućnost integracije u već postojeće web stranice, opciju preuzimanja aplikacije na mobilnim uređajima te planer za organizaciju budućih događaja.

- SLIKA 3 : prikaz sučelja za otvaranje natjecanja u DanceBUG aplikaciji

U usporedbi s DanceArenom, DanceBUG je znatno opširniji u svojim mogućnostima i namijenjen komercijalnoj uporabi te je korištena primarno u Sjevernoj Americi. Naša aplikacija je fokusirana na maloj skali lokalnih natjecanja. Pisana je na hrvatskom za što lakšu uporabu, podijeljena je na tri drugačija korisnička računa za jednostavnost te nudi trenutni pregled plasmana.

MOGUĆE NADOGRADNJE I PRILAGODBE PROJEKTOG ZADATKA

Nadogradnja aplikacije DanceArena može ići u dva smjera.

Prvi je širiti se unutar područja plesnih natjecanja, nalik na aplikaciju DanceBUG s kojom smo ju usporedili. Funkcije poput prijenosa uživo natjecanja za publiku ili novi tip korisničkog računa za plesne instruktore bi znatno proširili bazu korisnika te donijeli veću zaradu vlasnicima aplikacije.

Alternativni smjer nadogradnje bi bio proširiti tipove natjecanja na srodne sportove, poput gimnastike ili za organizaciju drugačijih događaja, poput zabava ili manifestacija. Ovakav smjer bi pretvorio DanceArenu u općenitiju aplikaciju za organizaciju i pregled događaja u okolini.

U oba slučaja je korisno nadodati funkcije poput podrške za drugačije jezike kako bi aplikacija bila korištena i van Hrvatske te verziju za mobitele za prijave i provjeru informacija od bilo gdje. Može se i nadodati sistem za chat unutar aplikacije kako bi komunikacija s organizatorima bila brža i jednostavnija.

OPSEG PROJEKTOG ZADATKA

Prva faza projekta je počela s raspodjelom uloga i dogovaranjem kako ćemo implementirati zadane specifikacije. Zatim smo odredili funkcionalne i nefunkcionalne zahtjeve aplikacije tako da pokriju sve zadatke i mogućnosti aplikacije te sve aktore i njihove mogućnosti u sustavu. Onda smo pomoću uml dijagrama i obrazaca uporabe prikazali ponašanje sustava, u kojima smo razradili implementaciju prije navedenih funkcionalnih zahtjeva i aktora koji ih koriste. Za najvažnije i kompliciranije funkcije smo napravili i sekvencijski dijagram radi lakšeg razumijevanja. Za kraj smo napravili bazu podataka i krenuli s implementacijom navedenih funkcija te uspješno deployali prvu verziju aplikacije s ograničenim sposobnostima.

FUNKCIJSKI ZAHTJEVI

ID	Opis	Prioritet	Izvor	Kriteriji prihvatanja
FZ1	Sustav omogućuje korisnicima prijavu preko Google računa putem OAuth 2.0 protokola.	Visok	Zahtjev dionika	Korisnik se može prijaviti putem Google računa i nakon prijave dobiva pristup svom profilu.
FZ2	Administrator može postaviti cijenu članstva, upravljati korisnicima te nadgledati objave i prijave.	Srednji	Zahtjev dionika	Administrator može ažurirati cijene, deaktivirati korisnike i pregledavati sustavske logove.
FZ3	Sustav omogućuje organizatoru stvaranje i uređivanje natjecanja.	Visok	Zahtjev dionika	Organizator može otvoriti natjecanje s datumom, lokacijom i opisom te pregledati i mijenjati prijave.
FZ3.1	Organizator bira suce za natjecanje.	Visok	Zahtjev dionika	Sustav onemogućava odabir parnog broja sudaca ili manje od tri suca.
FZ3.2	Organizator može pozvati neprijavljenog suca putem e-maila.	Srednji	Zahtjev dionika	Nakon unosa e-mail adrese suca, automatski se šalje poziv za registraciju.
FZ3.3	Organizator može zaključati prijave za natjecanje.	Visok	Zahtjev dionika	Nakon zaključavanja više nije moguće prijavljivanje ili uređivanje nastupa.
FZ4	Sustav omogućuje generiranje startne liste u PDF formatu nakon zaključavanja prijava.	Visok	Zahtjev dionika	Generirani PDF sadrži sve nastupe grupirane po kategorijama i može se preuzeti.
FZ5	Voditelj kluba može prijaviti svoje grupe i nastupe na otvorena natjecanja.	Visok	Zahtjev dionika	Voditelj može unijeti podatke o koreografiji, trajanje, koreografa i uploadati glazbu.

ID	Opis	Prioritet	Izvor	Kriteriji prihvatanja
FZ5.1	Voditelj kluba prilikom prijave mora priložiti glazbenu datoteku koja će se koristiti za nastup.	Visok	Zahtjev dionika	Sustav ne dopušta završetak prijave bez uspješnog učitavanja glazbene datoteke.
FZ6	Sustav omogućuje plaćanje kotizacija i članarina putem kreditne kartice.	Visok	Zahtjev dionika	Nakon plaćanja, sustav bilježi uspješnu transakciju i potvrđuje prijavu.
FZ7	Suci mogu tijekom natjecanja unositi ocjene za svaki nastup.	Visok	Zahtjev dionika	Sustav bilježi ocjene svih sudaca (0–30 bodova) i sprema ih uz nastup.
FZ8	Sustav automatski izračunava ukupne bodove i generira poredak po kategorijama.	Visok	Zahtjev dionika	Nakon završetka natjecanja, prikazuje se poredak s najboljim nastupima po kategoriji.
FZ9	Rezultati natjecanja se spremaju i može se izabrati s kojeg će korisnik pregledati javno dostupne rang liste, dok ocjene mogu vidjeti samo voditelji svojih grupa.	Visok	Zahtjev dionika	Javni korisnik vidi samo poredak, a prijavljeni voditelj vidi i bodove svog tima.
FZ10	Sustav omogućuje promjenu vizualne teme sučelja.	Nizak	Zahtjev sustava	Korisnik može odabrati drugu temu (npr. svijetla/tamna pozadina), a odabir se pamti i primjenjuje pri ponovnoj prijavi.

NEFUNKCIJSKI ZAHOTJEVI

ID	Opis	Prioritet	Kategorija
NFZ1	Sustav mora biti responzivan i prilagođen korištenju na svim veličinama ekrana (desktop, tablet, mobilni uređaji).	Visok	Upotrebljivost
NFZ2	Svaki mrežni zahtjev klijentske aplikacije mora biti obrađen unutar 1 sekunde.	Visok	Performanse
NFZ3	Baza podataka mora biti normalizirana do treće normalne forme (3NF).	Srednji	Održavanje
NFZ4	Sustav mora imati implementirane postavke pristupačnosti (npr. čitljivost, kontrast, podrška za čitače ekrana).	Nizak	Upotrebljivost

ID	Opis	Prioritet	Kategorija
NFZ5	Sustav mora podržavati najmanje 100 istovremenih korisnika bez pada performansi.	Srednji	Performanse

DIONICI

Organizatori - Otvaraju natjecanja, upravljaju prijavama i biraju suce

Vlasnici klubova - Prijavljaju grupe na natjecanja

Suci - Ocjenjuju nastupe na natjecanjima

Neregistrirani korisnici (gledatelji) - Mogu vidjeti rang listu natjecanja

Administratori - Održavaju i nadgledaju platformu te određuju cijenu članstva

Aktori i njihovi funkcionalni zahtjevi

A-1 Organizatori (inicijatori)

- **Funkcionalnost 1:** Prijava kao organizator (FZ1).
- **Funkcionalnost 2:** Kreiranje i upravljanje natjecanjima te upravljanje prijavama (FZ3).
 - **Podfunkcionalnost 1:** Biranje suca za natjecanje (FZ3.1).
 - **Podfunkcionalnost 2:** Pozivanje neprijavljenog suca pomoću e-maila (FZ3.2).
 - **Podfunkcionalnost 3:** Zaključavanje prijava za natjecanje (FZ3.3).
- **Funkcionalnost 3:** Preuzimanje pdf dokumenta startne liste nakon zaključavanja prijava (FZ4).
- **Funkcionalnost 4:** Plaćanje članarine (FZ6).
- **Funkcionalnost 5:** Pregled rang liste natjecanja (FZ9).
- **Funkcionalnost 6:** Promjena vizualne teme sučelja (FZ10).

A-2 Voditelji kluba (inicijatori)

- **Funkcionalnost 1:** Prijava kao voditelj kluba (FZ1).
- **Funkcionalnost 2:** Preuzimanje pdf dokumenta startne liste nakon zaključavanja prijava (FZ4).
- **Funkcionalnost 3:** Prijava grupa na otvorena natjecanja (FZ5).
 - **Podfunkcionalnost 1:** Prilaganje glazbe koja će se koristiti za nastup (FZ5.1).
- **Funkcionalnost 4:** Plaćanje kotizacije za natjecanja (FZ6).
- **Funkcionalnost 5:** Pregled rang liste natjecanja i osvojenih bodova svoje grupe (FZ9).
- **Funkcionalnost 6:** Promjena vizualne teme sučelja (FZ10).

A-3 Suci (inicijatori)

- **Funkcionalnost 1:** Prijava kao sudac (FZ1).
- **Funkcionalnost 2:** Preuzimanje pdf dokumenta startne liste nakon zaključavanja prijava (FZ4).
- **Funkcionalnost 3:** Unošenje ocjena za nastupe na natjecanjima na kojima sude (FZ7).
- **Funkcionalnost 4:** Pregled rang liste natjecanja (FZ9).
- **Funkcionalnost 5:** Promjena vizualne teme sučelja (FZ10).

A-4 Neregistrirani korisnik / Gledatelji (sudionici)

- **Funkcionalnost 1:** Pregled rang liste natjecanja (FZ9).
- **Funkcionalnost 2:** Promjena vizualne teme sučelja (FZ10).

A-5 Administratori (sudionici)

- **Funkcionalnost 1:** Prijava kao administrator (FZ1).
- **Funkcionalnost 2:** Postavljanje cijene članstva, upravljanje korisnicima te nadgledanje objava i prijava (FZ2).
- **Funkcionalnost 3:** Pregled rang liste natjecanja (FZ9).
- **Funkcionalnost 4:** Promjena vizualne teme sučelja (FZ10).

Obrasci uporabe

Opis obrazaca uporabe

UC1 - Prijava i registracija korisnika

- Glavni sudionik: Organizator/Voditelj kluba/Sudac
- Cilj: Siguran pristup sustavu i korisničkom računu
- Sudionici: Autentifikacijski servis (Google OAuth 2.0), Baza podataka
- Preduvjet: /
- Opis osnovnog tijeka:

1. Prijava mailom putem vanjskog servisa. (FZ1: Prijava preko OAuth 2.0 protokola)
2. Registrirani korisnik preusmjeren je na svoju stranicu, novog korisnika se traži unos dodatnih podataka i uloge.

- Opis mogućih odstupanja:

- 1A: Korisnik ne uspijeva autorizaciju na vanjskom servisu -> Sustav prikazuje poruku o pogrešci
- 2A: Korisnik ne ispuni obavezna dodatna polja -> sustav javlja pogrešku i traži unos prije nastavka

UC2 - Upravljanje natjecanjima

- Glavni sudionik: Organizator
- Cilj: Kreiranje, uređivanje i brisanje plesnih natjecanja
- Sudionici: Baza podataka
- Preduvjet: /
- Opis osnovnog tijeka:

1. Organizator kreira/uređuje (UC2.1 i UC2.2) natjecanje ispunjavanjem obrasca (FZ3: Stvaranje i uređivanje natjecanja).
2. Organizator prihvaća i uređuje prijavljene nastupe (UC2.3).
3. Organizator zaključava prijave natjecanja (UC2.4) i sustav generira listu natjecatelja (FZ4: Generiranje startne liste).

- Opis mogućih odstupanja:

- 1A: Nedostaju obavezni podaci u obrascu -> sustav upozorava i onemogućuje spremanje.
- 1B: Neispravan format datuma/ vremena -> sustav traži ispravak.
- 2A: Plaćanje kotizacije nije potvrđeno -> sustav označava prijavu kao nepotpunu

UC3 - Prijava plesnih nastupa

- Glavni sudionik: Voditelj kluba
- Cilj: Prijava plesnih parova ili grupa na natjecanja te pregled i izmjenu postojećih prijava
- Sudionici: Baza podataka
- Preduvjet: Postoji barem jedno aktivno natjecanje otvoreno za prijave

- Opis osnovnog tijeka:

1. Voditelj kluba stvara novi nastup ispunjavanjem obrasca (UC3.1 i UC3.2).
2. Voditelj prijavljuje nastup na odabrano natjecanje te plaća postavljenu kotizaciju (FZ5: Prijava nastupa na otvorena natjecanja, FZ6: Plaćanje kotizacije i članarine).

- Opis mogućih odstupanja:

- 1A: Voditelj ne unosi sve potrebne podatke u opis nastupa -> sustav prikazuje grešku.
- 2A: Plaćanje kotizacije ne uspije (npr. odbijena kartica) -> prijava se ne sprema, sustav traži ponovni pokušaj.
- 2B: Natjecanje je zaključano ili pun kapacitet -> sustav zabranjuje novu prijavu.

UC4 - Ocjenjivanje nastupa

- Glavni sudionik: Sudac
- Cilj: Unos i potvrda ocjena za nastupe u natjecanju
- Sudionici: Baza podataka
- Preduvjet: Sudac je prijavljen na natjecanju
- Opis osnovnog tijeka:

1. Sudac dodijeljen natjecanju ima pregled svih nastupa.
2. Po završetku nastupa, sudac ga ocjenjuje ocjenom 1-30 (FZ7: Unos ocjena).
3. Sustav generira rang listu (UC4.1) (FZ8: Generiranje rang liste).

- Opis mogućih odstupanja:

- 1A: Sudac nije dodijeljen natjecanju -> sustav ne dopušta pregled nastupa.
- 2A: Sudac pokuša unijeti ocjenu izvan raspona 1–30 -> sustav odbija unos i traži ispravak.
- 2B: Sudac pokuša ocijeniti nastup više puta -> sustav javlja da je ocjena već spremljena.

UC5 - Pregled rezultata i rang lista

- Glavni sudionik: Organizator/Voditelj kluba/Sudac/Neregistrirani korisnik (Gledatelji)
- Cilj: Pregled rezultata i rang lista održanih natjecanja
- Sudionici: Baza podataka
- Preduvjet: Natjecanje je održano i njegova rang lista spremljena
- Opis osnovnog tijeka:

1. Svaki korisnik može pristupiti rang listi (FZ9: Pregled rang liste).
2. Voditeljima klubova sustav generira dodijeljene ocjene za sve nastupe (UC5.1).

- Opis mogućih odstupanja:

- 1A: Nema dodijeljenih ocjena za natjecanje -> sustav prikazuje poruku „Rang lista nije dostupna“.

UC6 - Upravljanje korisnicima

- Glavni sudionik: Administrator
- Cilj: Nadzor nad korisnicima sustava, promjena i deaktivacija korisničkih računa

- Sudionici: Baza podataka
- Preduvjet: /
- Opis osnovnog tijeka:

1. Administrator se prijavljuje u admin sučelje (FZ1: Prijava preko OAuth 2.0 protokola).
2. Administrator ima pregled svih korisnika i podataka u sustavu te mogućnost njihove izmjene (UC6.1 i UC6.2) (FZ2: Upravljanje korisnicima i postavljanje cijene članstva).

- Opis mogućih odstupanja:

- 1A: Administrator unese krive podatke za prijavu -> sustav odbija pristup.
- 2A: Sustav detektira nedozvoljeni pokušaj uklanjanja administratora samog sebe -> blokira radnju.

UC7 - Postavljanje cijene članstva

- Glavni sudionik: Administrator
- Cilj: Omogućiti postavljanje, izmjenu i ažuriranje cijene članstva
- Sudionici: /
- Preduvjet: /
- Opis osnovnog tijeka:

1. Administrator postavlja godišnju članarinu za organizatore na svojoj platformi (FZ2: Upravljanje korisnicima i postavljanje cijene članstva).

- Opis mogućih odstupanja:

- 1A: Uneseni iznos nije numerički -> sustav prikazuje pogrešku.
- 1B: Iznos je negativan -> sustav odbija unos.

UC8 - Prilagodba izgleda stranice

- Glavni sudionik: Organizator/Voditelj kluba/Sudac/Neregistrirani korisnik (Gledatelji)
- Cilj: Promjenu vizualne teme korisničkog sučelja
- Sudionici: /
- Preduvjet: /
- Opis osnovnog tijeka:

1. Korisnik odabire vizualnu temu za sučelje.
2. Sustav prikazuje sučelje prilagođeno temi (FZ10: Promjena vizualne teme sučelja).

- Opis mogućih odstupanja:

- 1A: Odabrana tema nije dostupna (npr. obrisana) -> sustav vraća zadanu temu.
- 2A: Spremanje teme na profil korisnika ne uspije -> sustav koristi privremenu temu samo za trenutnu sesiju.

UC9 - Kartično plaćanje

- Glavni sudionik: Voditelj kluba, Organizator
- Cilj: Plaćanje kotizacije i članarine

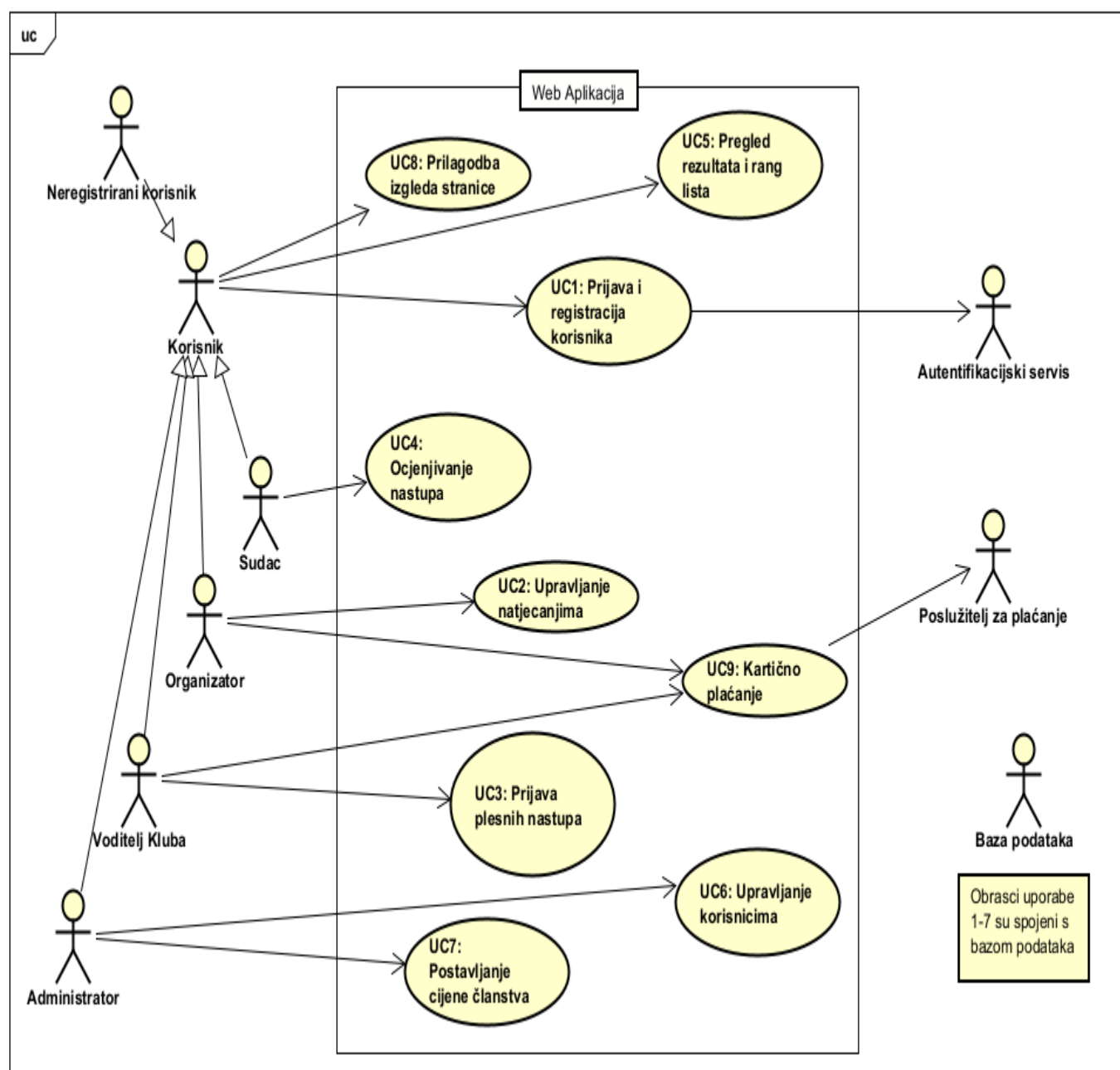
- Sudionici: Poslužitelj za plaćanje
- Preduvjet: /
- Opis osnovnog tijeka:

1. Voditelj kluba/organizator odabire plaćanje kreditnom karticom.
2. Sustav ga preusmjerava na vanjski servis(PayPal) putem kojeg se obavlja verifikacija korisnika i kreditne kartice (FZ6: Plaćanje kotizacije i članarine).

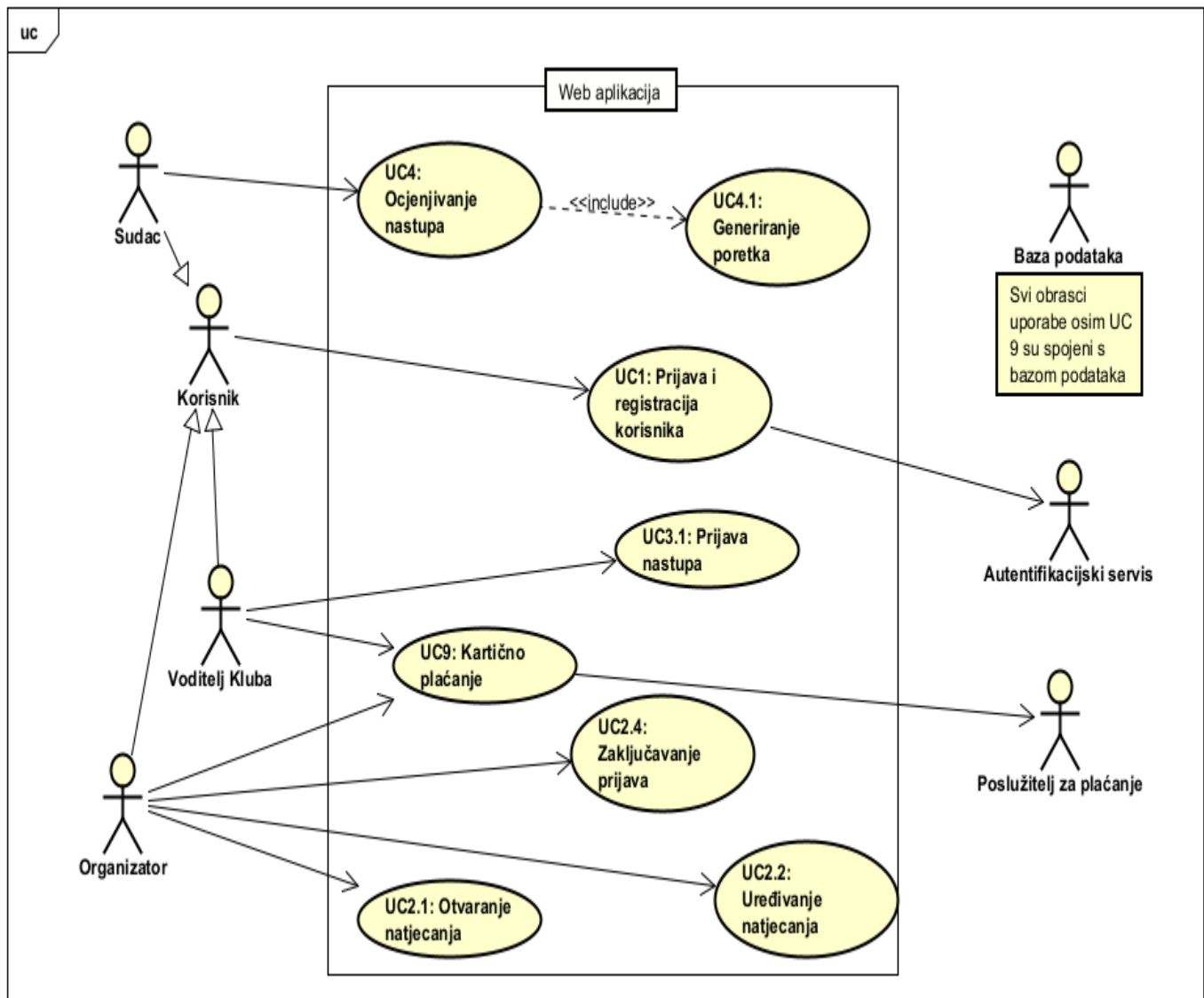
- Opis mogućih odstupanja:

- 1A: Sustav ne može uspostaviti vezu s vanjskim servisom (PayPal / kartice) -> prikazuje poruku „Pokušajte kasnije“.
- 1B: Podaci o kartici ne prođu verifikaciju -> plaćanje odbijeno -> prijava ili članarina nije aktivirana.
- 2A: Korisnik prekine proces plaćanja -> status ostaje „neplaćeno“.

1. Visokorazinski obrasci uporabe cijelog sustava

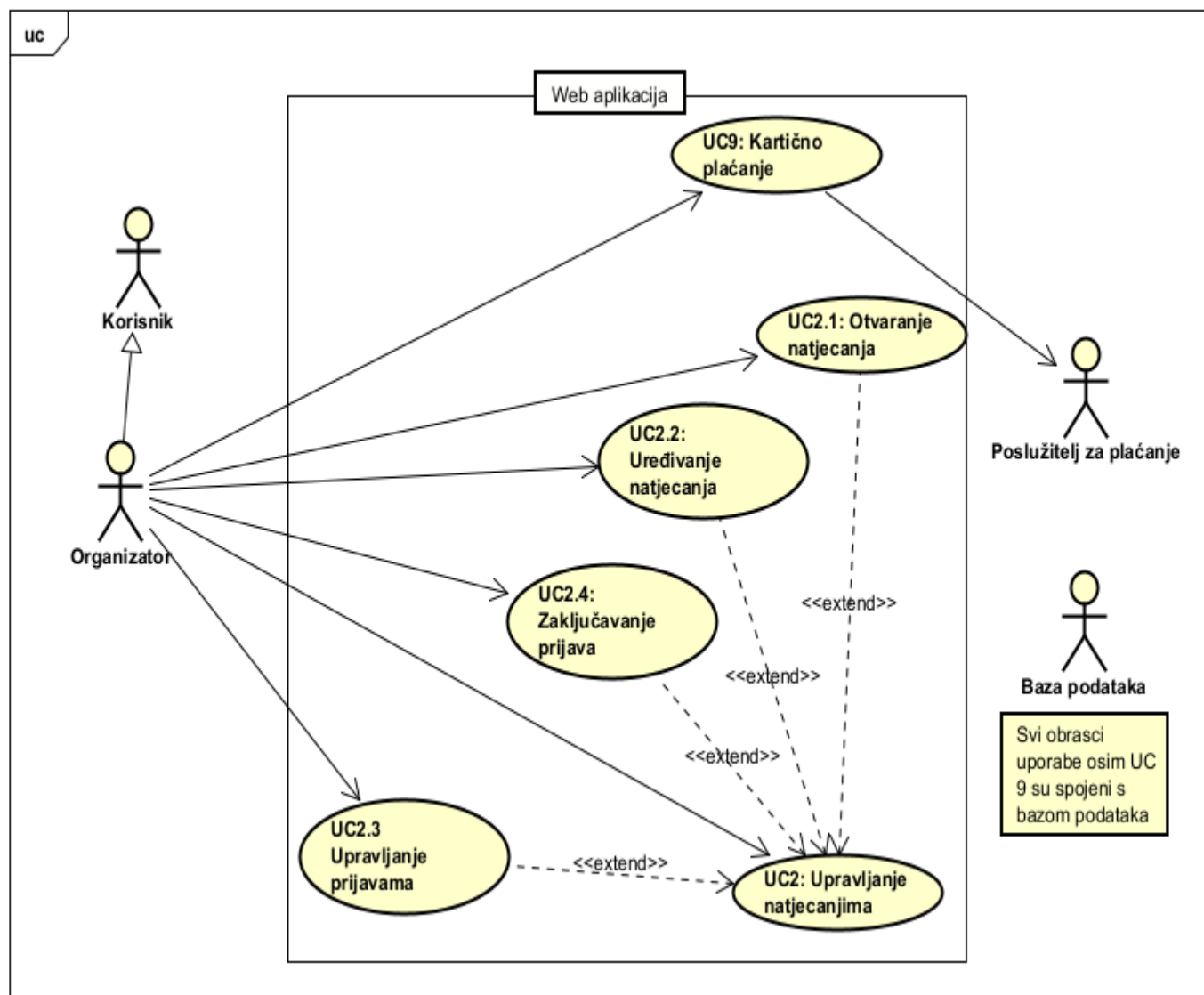


2. Obrasci uporabe za ključne funkcionalnosti

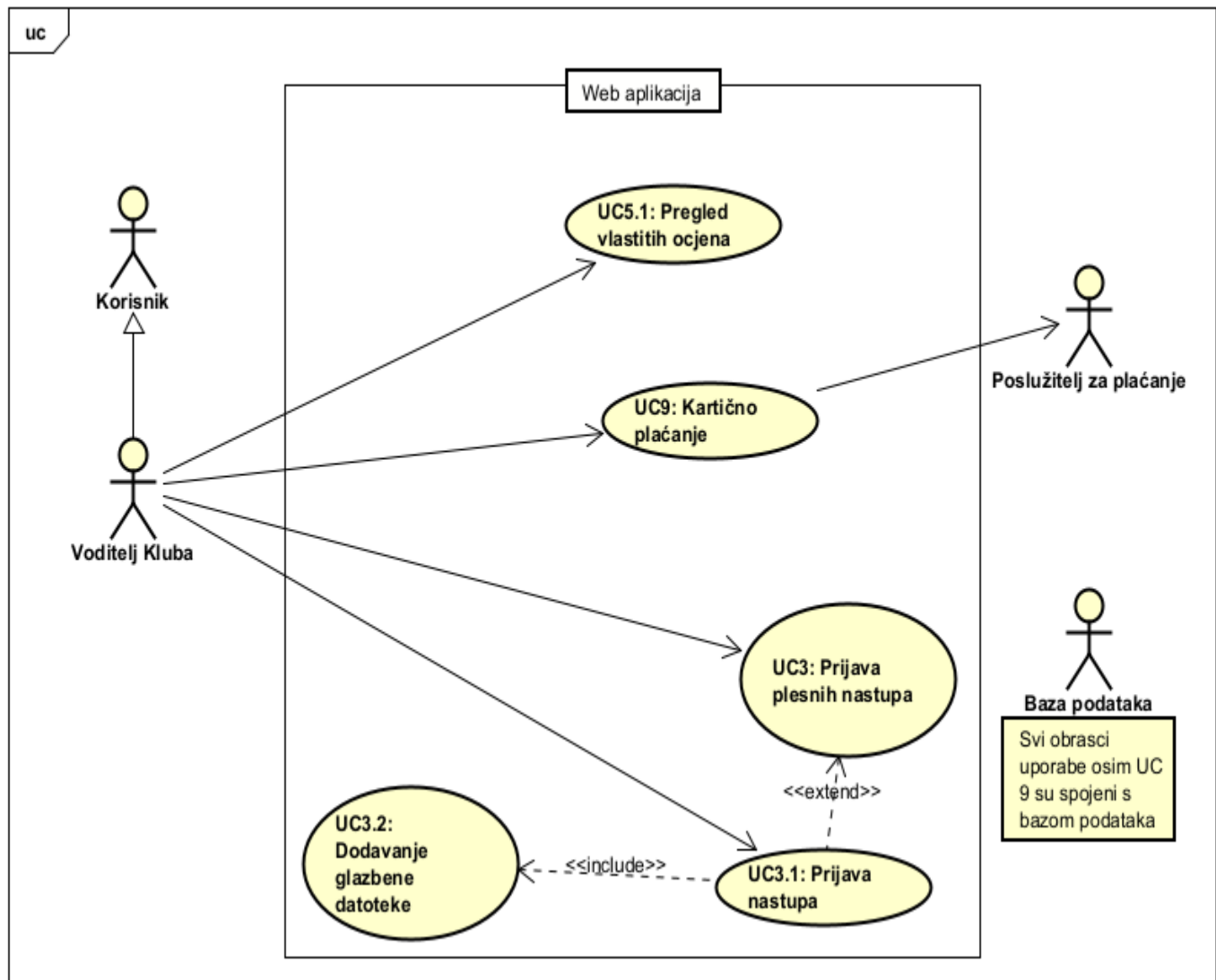


3. Obrasci uporabe za korisničke uloge

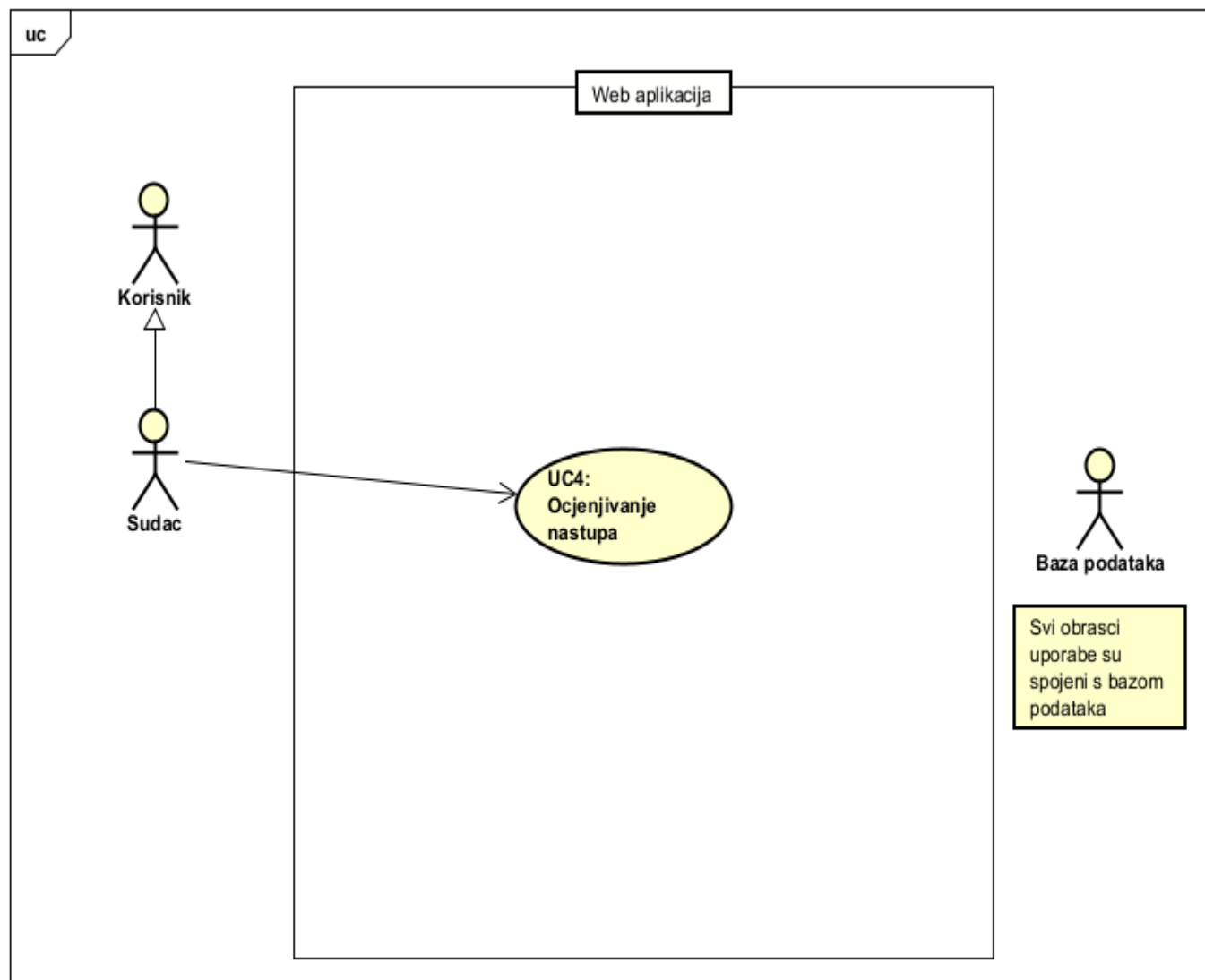
- Organizator



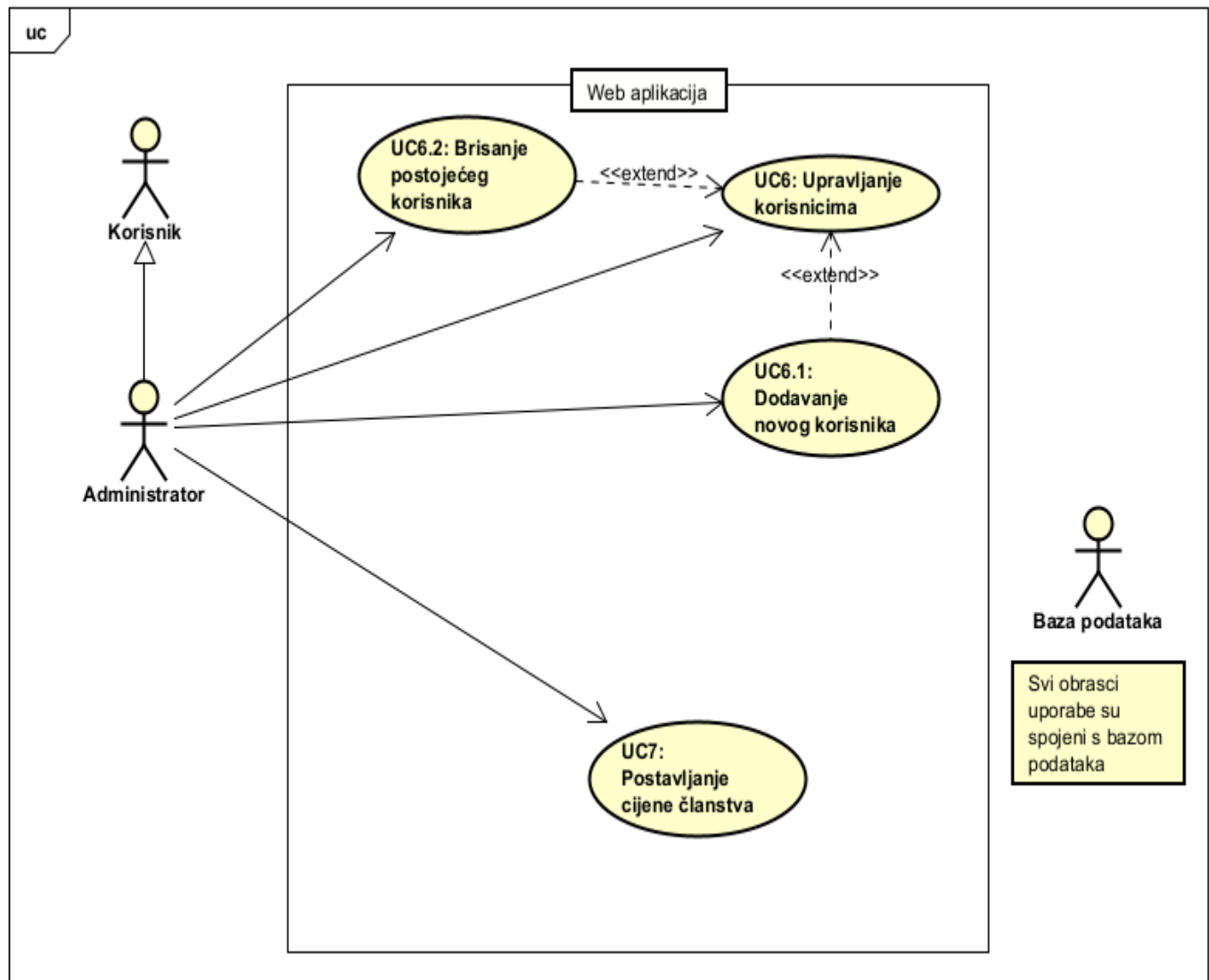
- Voditelj kluba



- Sudac



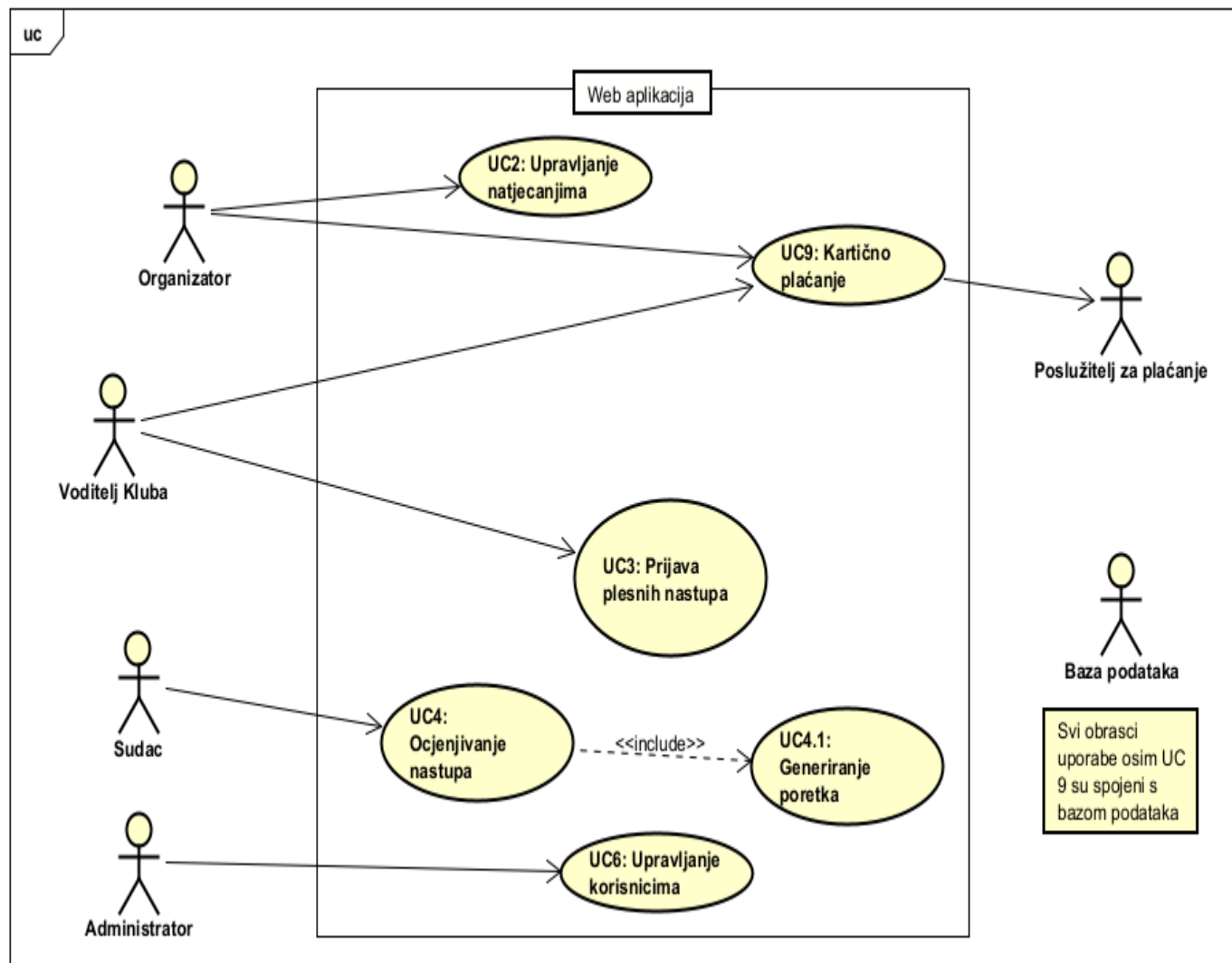
- Administrator



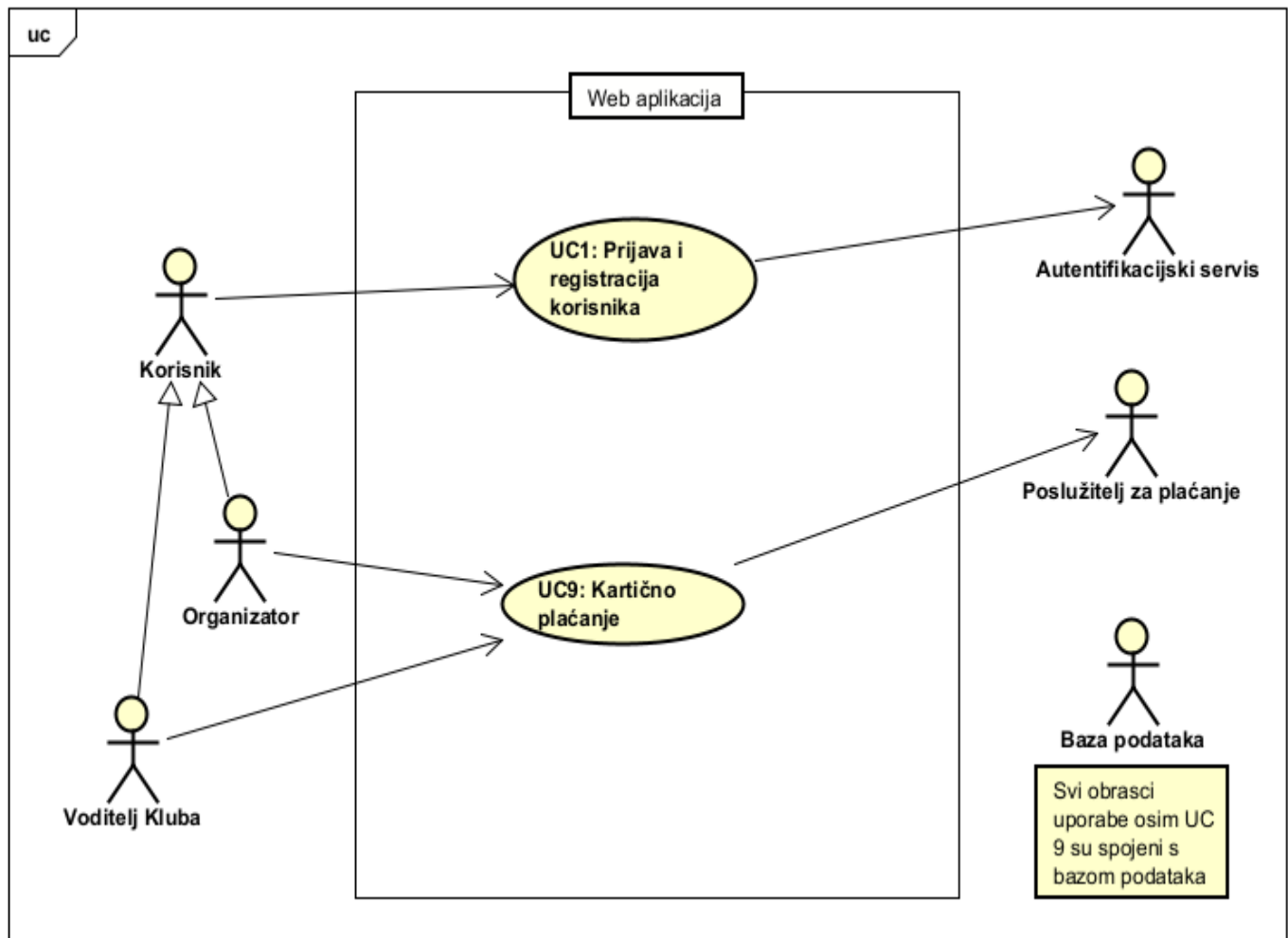
- Neregistrirani korisnik (Gledatelji)

Nemaju zasebne obrasce osim obrazaca aktora Korisnika

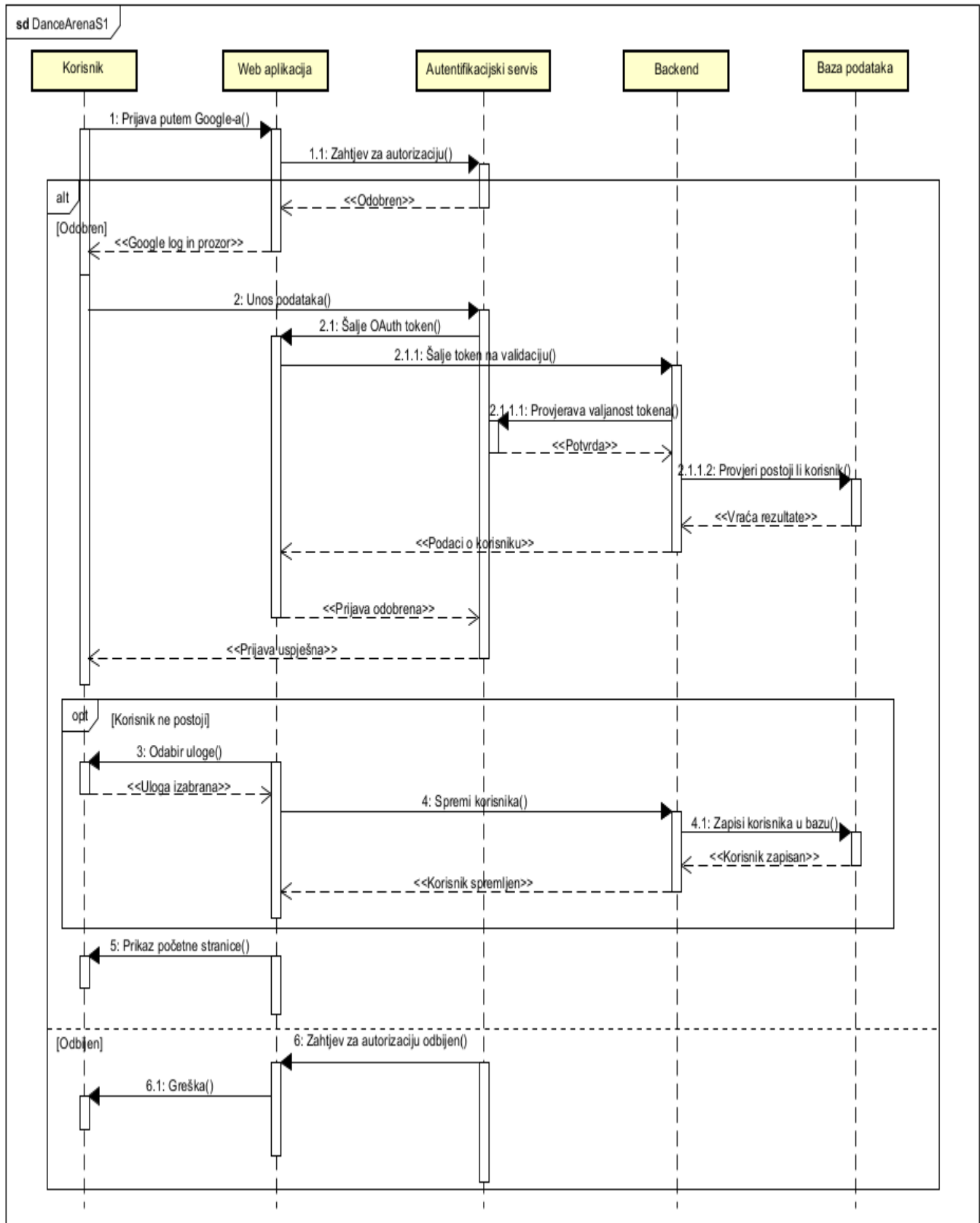
4. Obrasci uporabe za osnovne poslovne procese



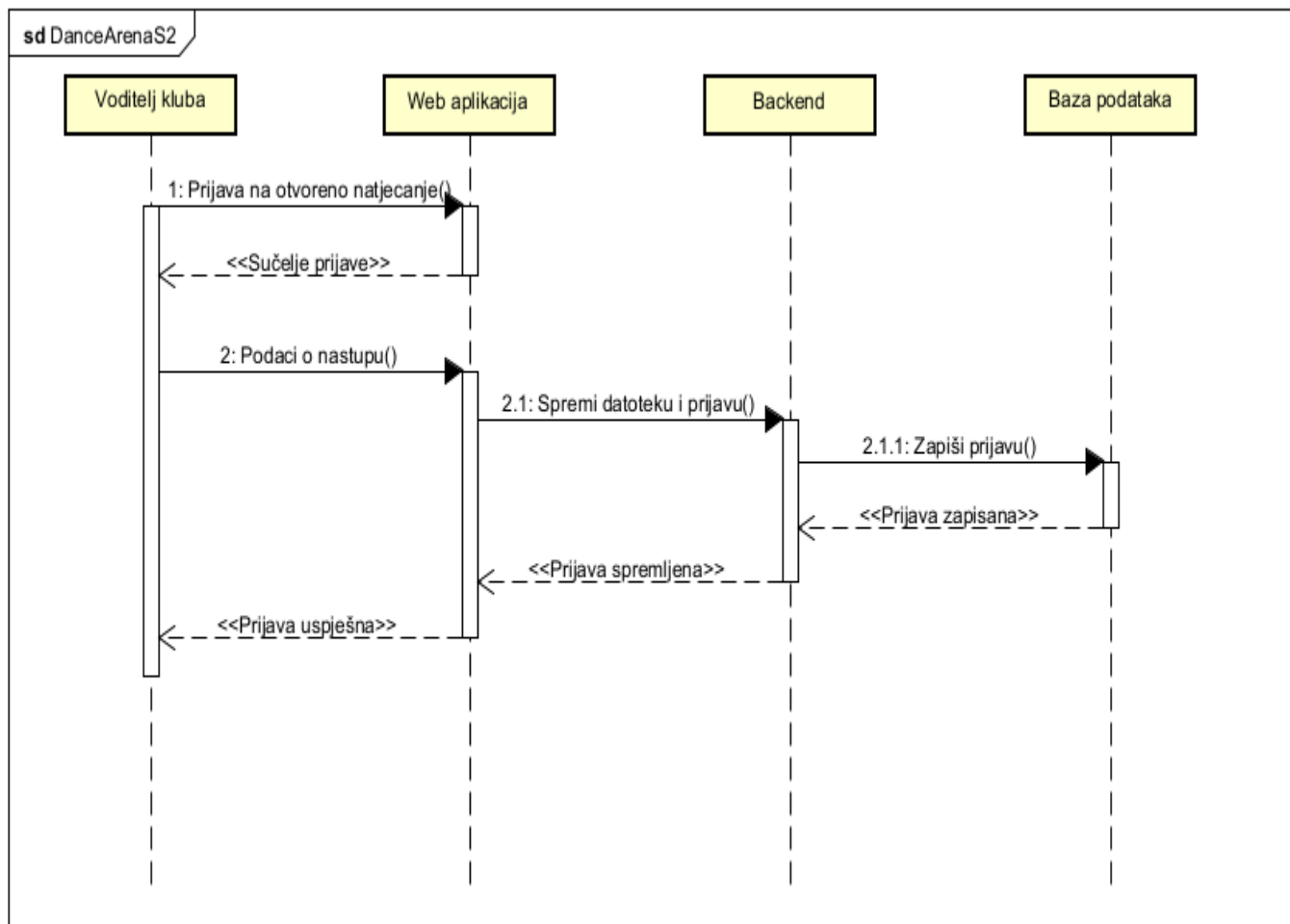
5. Obrasci uporabe za kritične sustave i integracije



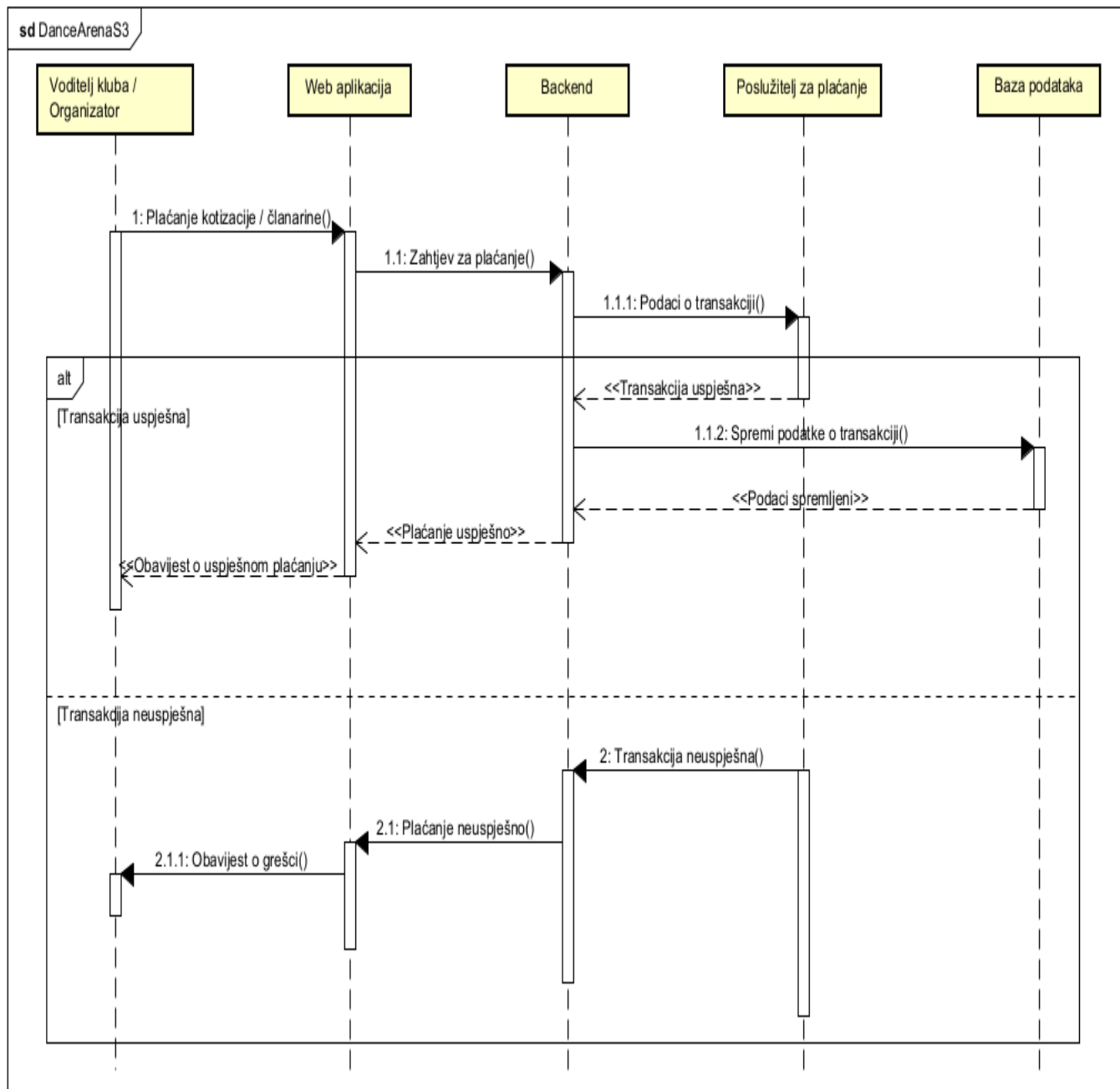
Sekvencijski dijagrami



- Prijava / registracija korisnika: Obuhvaća FZ1 - Prijava preko Google računa putem OAuth 2.0 protokola. Korisnik se prijavljuje i pritom sustav koristi OAuth 2.0. Ukoliko je dostupan, korisnik se prijavljuje upisujući podatke u njihov log in prozor. Zatim, OAuth šalje token koji će backend provjeriti radi sigurnosti. Zatim će backend provjeriti bazu podataka je li korisnik već registriran. Ako je, biti će mu prikazana njegova početna stranica. Ako korisnik nije prisutan u bazi podataka, morat će izabrati svoju ulogu te će njegov račun biti pohranjen u bazi



- Prijava nastupa na otvorena natjecanja: Obuhvaća FZ5 - Prijava grupa i nastupa na otvorena natjecanja i FZ5.1: Prilaganje glazbene datoteke za nastup. Voditelj kluba izabire otvoreno natjecanje i čeka odgovor od aplikacije. Zatim upisuje podatke o svom nastupu i prilaže glazbenu datoteku te se to sprema u backend. Backend zapisuje prijavu u bazu i javlja frontendu kad je sve obavljeno te frontend obavještava voditelja kluba da je prijava uspješna



- Plaćanje Obuhvaća FZ6 - Plaćanje kotizacija i članarina putem kreditne kartice. Voditelj kluba ili organizator idu platiti kotizaciju / članarinu te aplikacija šalje zahtjev za plaćanje backendu, koji proslijeđuje informacije o plaćanju Paypalu. Ako je transakcija uspješna, backend to zapisuje u bazu podataka te javlja aplikaciji koja obavještava voditelja kluba / organizatora

U svrhu učinkovitosti pri razvoju aplikacije odlučili smo se za arhitekturu klijent–poslužitelj. Sustav možemo podijeliti na frontend i backend, koji se zatim mogu razvijati i nadograđivati neovisno jedan o drugome. Za backend koristimo Python i Django. Python jer je jednostavan objektno orijentiran jezik s puno ugrađenih funkcionalnosti, a Django jer pojednostavljuje razvoj backend-a i automatski ostvaruje neke funkcionalnosti (admin sučelje i bazu podataka bez da programer koristi SQL). Za backend smo koristili React, Vite i Figma. React jer je najkorištenija javascript biblioteka za izgradnju klijentskog djela bazirana na komponentama kojima se omogućava brza i jednostavna izgradnja sustava. Vite jer je alat za razvoj koji pruža iznimno brzo pokretanje i trenutno osvježavanje onog dijela koda koji izmjenjen bez potrebe za osvježavanjem cijele stranice. Figma jer je besplatan alat za kolaborativan rad na dizajniranju korisničkog sučelja. Sustav koristi OAuth 2.0 protokol kao vanjski servis za autentifikaciju korisnika. Ovim pristupom omogućuje se sigurna prijava korisnika putem postojećih Google korisničkih računa, bez potrebe za pohranom lozinki unutar samog sustava. Time se postiže viša razina zaštite osobnih podataka i pojednostavljuje proces prijave. Za obradu financijskih transakcija (plaćanje članarina i kotizacija) sustav se integrira s PayPal servisom. PayPal omogućuje pouzdano i standardizirano kartično plaćanje te automatsku potvrdu uspješnih transakcija, čime se osigurava točna evidencija uplata i transparentnost financijskih procesa unutar sustava.

Baza podataka

Tablica	Opis
users_user	Svi korisnici sustava (organizatori, voditelji klubova, suci i administratori) s osnovnim podacima i ulogom.
users_organizersubscription	Evidencija plaćanja članarine organizatora i status aktivnog članstva.
users_organizersubscriptionprice	Povijest i trenutačna cijena članarine organizatora.
competitions_competition	Informacije o plesnim natjecanjima (naziv, datum, lokacija, status, kotizacija).
competitions_appearance	Pojedinačni nastupi prijavljeni na natjecanja (koreografija, trajanje, glazba, kategorije, status prihvaćenosti).
competitions_agecategory	Dobne kategorije natjecanja (djeca, juniori, seniori).
competitions_stylecategory	Stilovi plesa (balet, hip hop, jazz, step, break).
competitions_groupsizecategory	Veličina plesne grupe (solo, duo, mala grupa, formacija).
competitions_competitionjudge	Poveznica između natjecanja i sudaca koji sudjeluju u ocjenjivanju.
competitions_grade	Ocjene koje su suci dodijelili pojedinim nastupima.
competitions_result	Konačni rezultati natjecanja i poredak nastupa po kategorijama.

Tablica	Opis
competitions_competition_age_categories	Relacijska tablica između natjecanja i dobnih kategorija.
competitions_competition_style_categories	Relacijska tablica između natjecanja i stilova plesa.
competitions_competition_group_size_categories	Relacijska tablica između natjecanja i veličina plesnih grupa.
competitions_mediafile	Medijske datoteke (npr. glazba za nastupe) pohranjene u sustavu.
account_emailaddress	Evidencija e-mail adresa korisnika i status njihove potvrde.
account_emailconfirmation	Podaci o potvrdi e-mail adresa korisnika.
authtoken_token	Tokeni za API autentifikaciju korisnika.
socialaccount_socialaccount	Poveznica korisničkih računa s vanjskim OAuth 2.0 servisima (npr. Google).
social_auth_usersocialauth	Podaci o autentifikaciji korisnika putem vanjskih servisa.

Opći opis

Baza podataka sustava za plesna natjecanja implementirana je u **PostgreSQL-u** i služi za pohranu korisničkih podataka, autentifikacije, natjecanja, prijava nastupa, ocjenjivanje, rezultata te pretplata organizatora. Struktura baze temelji se na Django ORM-u te koristi dodatne pakete za autentifikaciju i OAuth prijavu.

Baza je podijeljena u sljedeće logičke cjeline:

- Korisnici i autentifikacija
- Natjecanja i nastupi
- Ocjenjivanje i rezultati
- Medijski sadržaj
- Pretplate organizatora

Korisnici i autentifikacija

users_user

Atribut	Opis
id	Jedinstveni identifikator korisnika
username	Korisničko ime
email	Email adresa korisnika
password	Hashirana lozinka

Atribut	Opis
first_name	Ime korisnika
last_name	Prezime korisnika
role	Uloga korisnika (ORGANIZER, CLUB_MANAGER, JUDGE, ADMIN)
club_name	Naziv kluba
club_location	Lokacija kluba
contact	Kontakt telefon
is_active	Status aktivacije računa
is_superuser	Administratorske ovlasti
date_joined	Datum registracije

account_emailaddress

Atribut	Opis
id	Identifikator zapisa
email	Email adresa
verified	Oznaka je li email potvrđen
primary	Oznaka primarnog emaila
user_id	Poveznica na korisnika

authtoken_token

Atribut	Opis
key	Token za API autentifikaciju
created	Vrijeme kreiranja tokena
user_id	Korisnik kojem token pripada

Natjecanja

competitions_competition

Atribut	Opis
id	Identifikator natjecanja
name	Naziv natjecanja
date	Datum održavanja

Atribut	Opis
location	Lokacija natjecanja
description	Opis natjecanja
status	Status (draft, active, closed)
registration_fee	Kotizacija
organizer_id	Organizator natjecanja
starting_list_id	Startna lista (PDF)

competitions_appearance

Atribut	Opis
id	Identifikator nastupa
choreography	Naziv koreografije
length	Trajanje nastupa
choreograph	Autor koreografije
paid_registration	Status plaćanja
accepted	Status prihvaćenosti
competition_id	Natjecanje
club_manager_id	Voditelj kluba
age_category_id	Dobna kategorija
style_category_id	Plesni stil
group_size_category_id	Veličina grupe
music_id	Priložena glazba

competitions_agecategory

Atribut	Opis
id	Identifikator kategorije
name	Naziv dobne kategorije

competitions_stylecategory

Atribut	Opis
id	Identifikator stila

Atribut	Opis
name	Naziv plesnog stila

competitions_groupsizecategory

Atribut	Opis
id	Identifikator
name	Tip nastupa (solo, duo, grupa)

Ocjenjivanje i rezultati

competitions_grade

Atribut	Opis
id	Identifikator ocjene
grade	Dodijeljena ocjena
appearance_id	Nastup
judge_id	Sudac

competitions_result

Atribut	Opis
id	Identifikator rezultata
rank	Ostvareni plasman
appearance_id	Nastup
competition_id	Natjecanje

competitions_competitionjudge

Atribut	Opis
id	Identifikator
competition_id	Natjecanje
judge_id	Sudac

Medijski sadržaj

competitions_mediafile

Atribut	Opis
id	Identifikator datoteke
title	Naziv datoteke
file_type	Tip datoteke
file	Putanja do datoteke
uploaded_at	Vrijeme prijensa

Pretplate organizatora

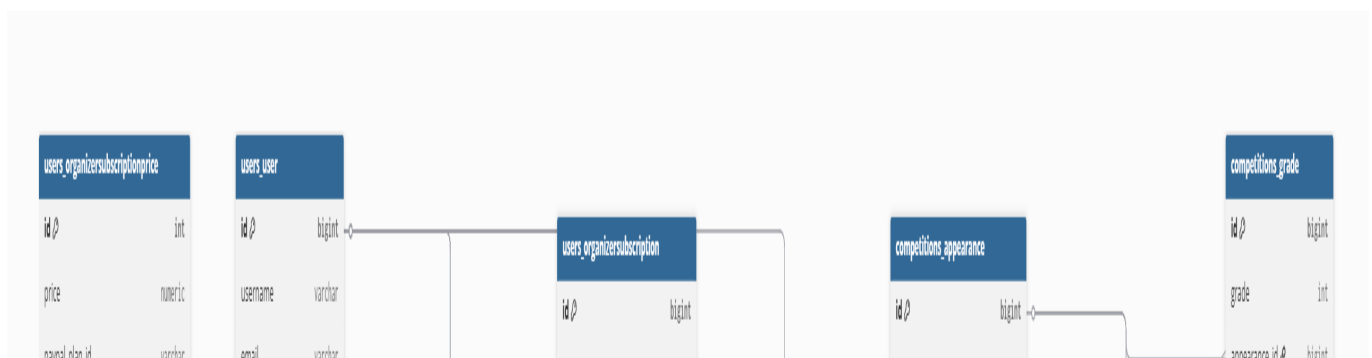
users_organizersubscription

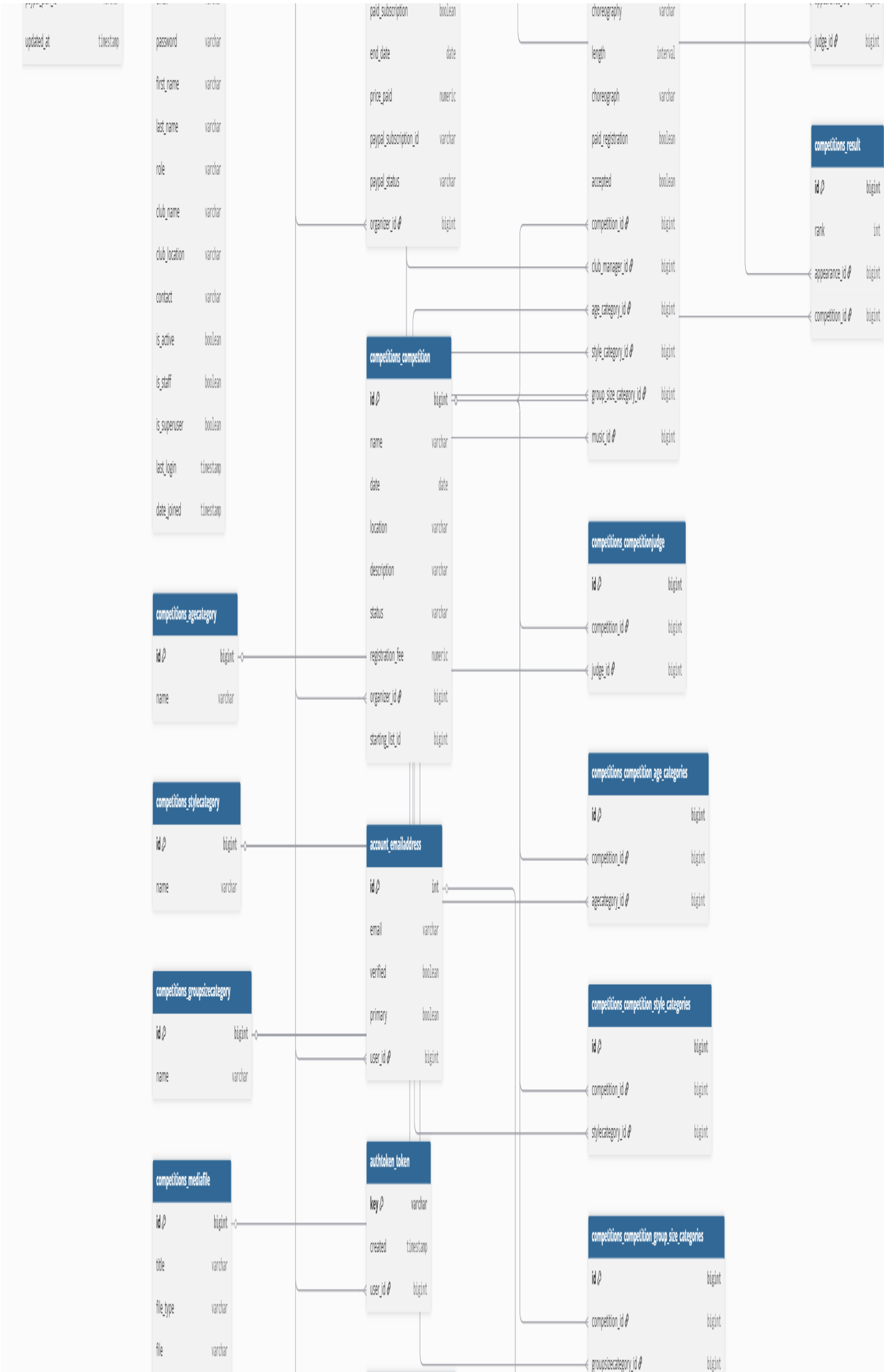
Atribut	Opis
id	Identifikator pretplate
paid_subscription	Status pretplate
end_date	Datum isteka
organizer_id	Organizator
price_paid	Plaćeni iznos
paypal_subscription_id	PayPal ID
paypal_status	Status PayPal pretplate

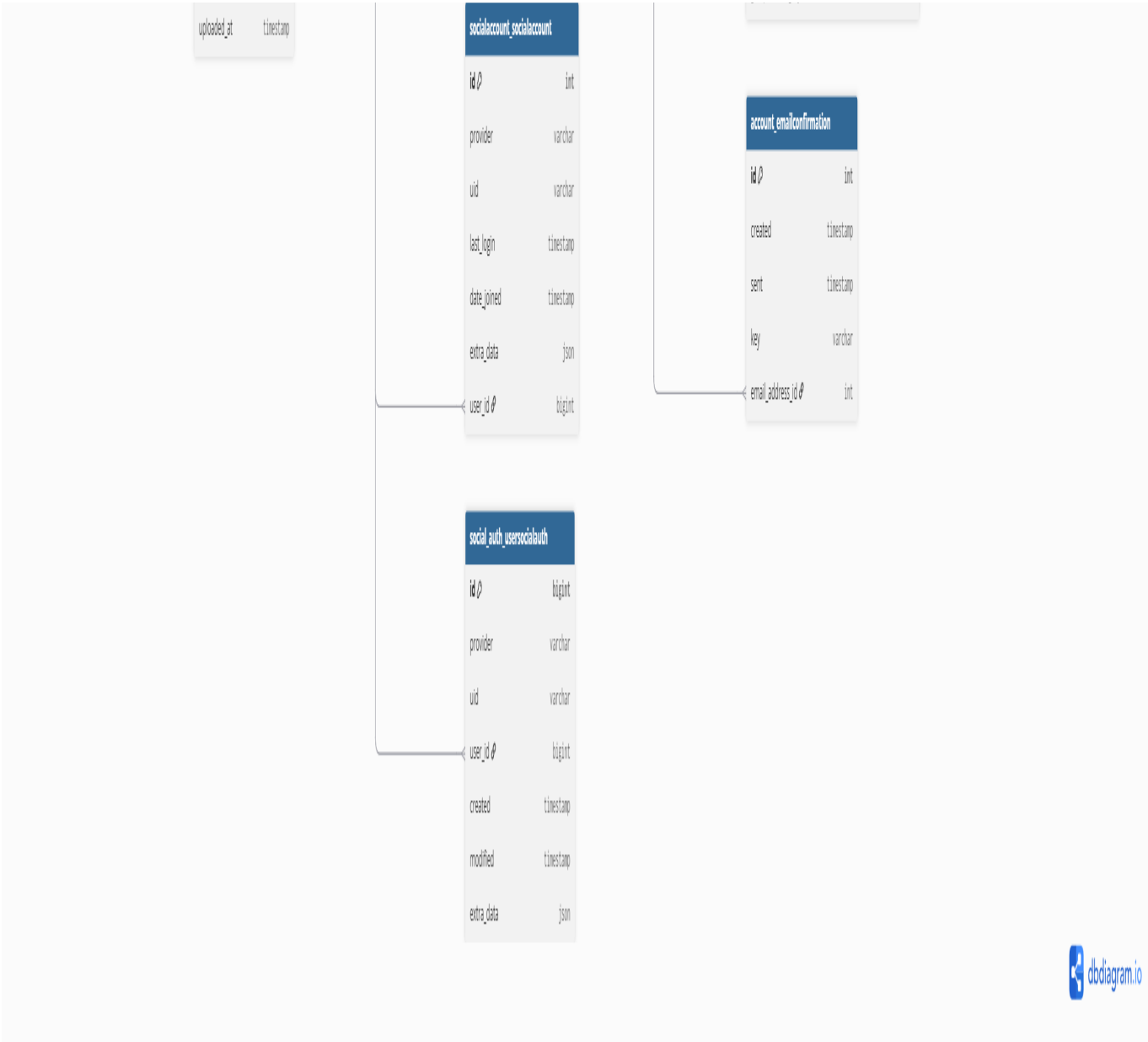
users_organizersubscriptionprice

Atribut	Opis
id	Identifikator
price	Iznos pretplate
paypal_plan_id	PayPal plan
updated_at	Zadnje ažuriranje

Dijagram baze podataka

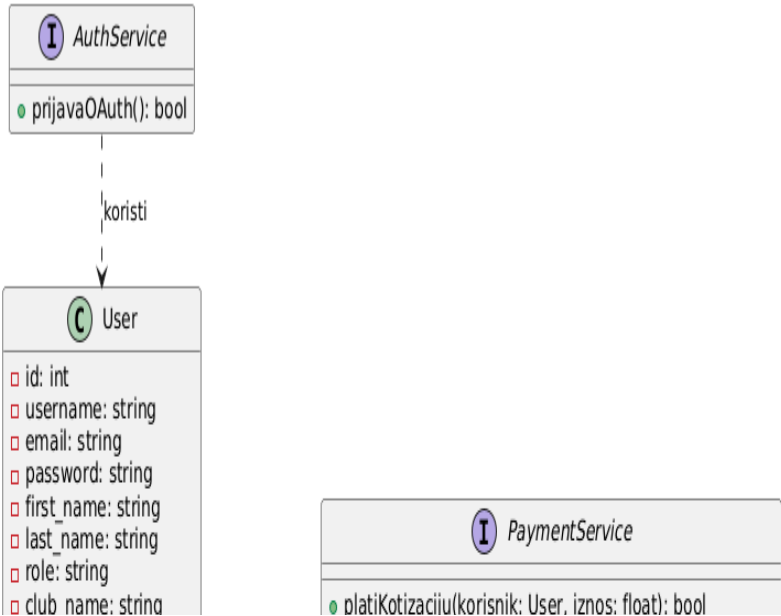




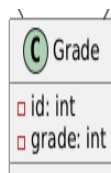


Dijagram razreda

UML Dijagram razreda - Sustav za plesna natjecanja

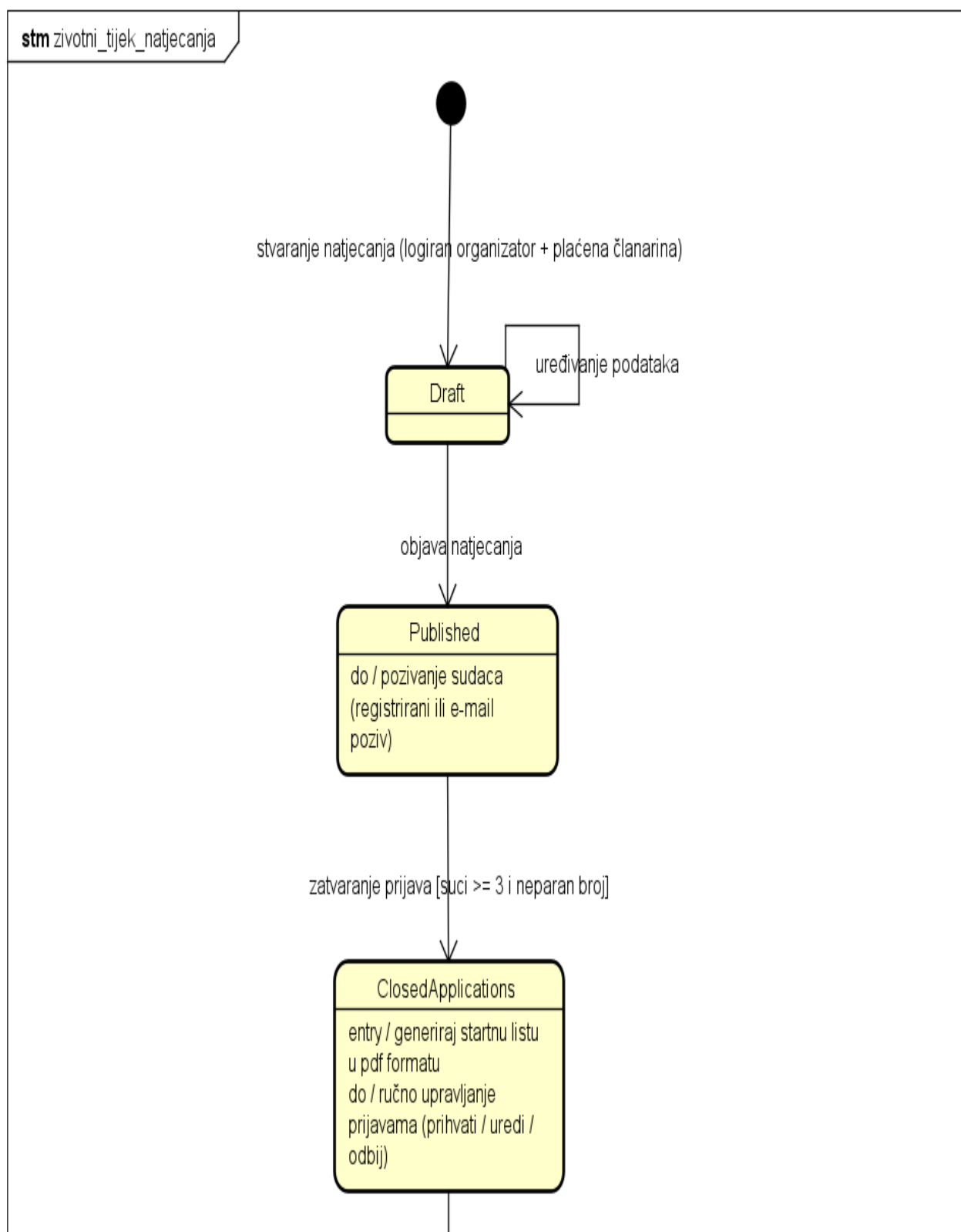


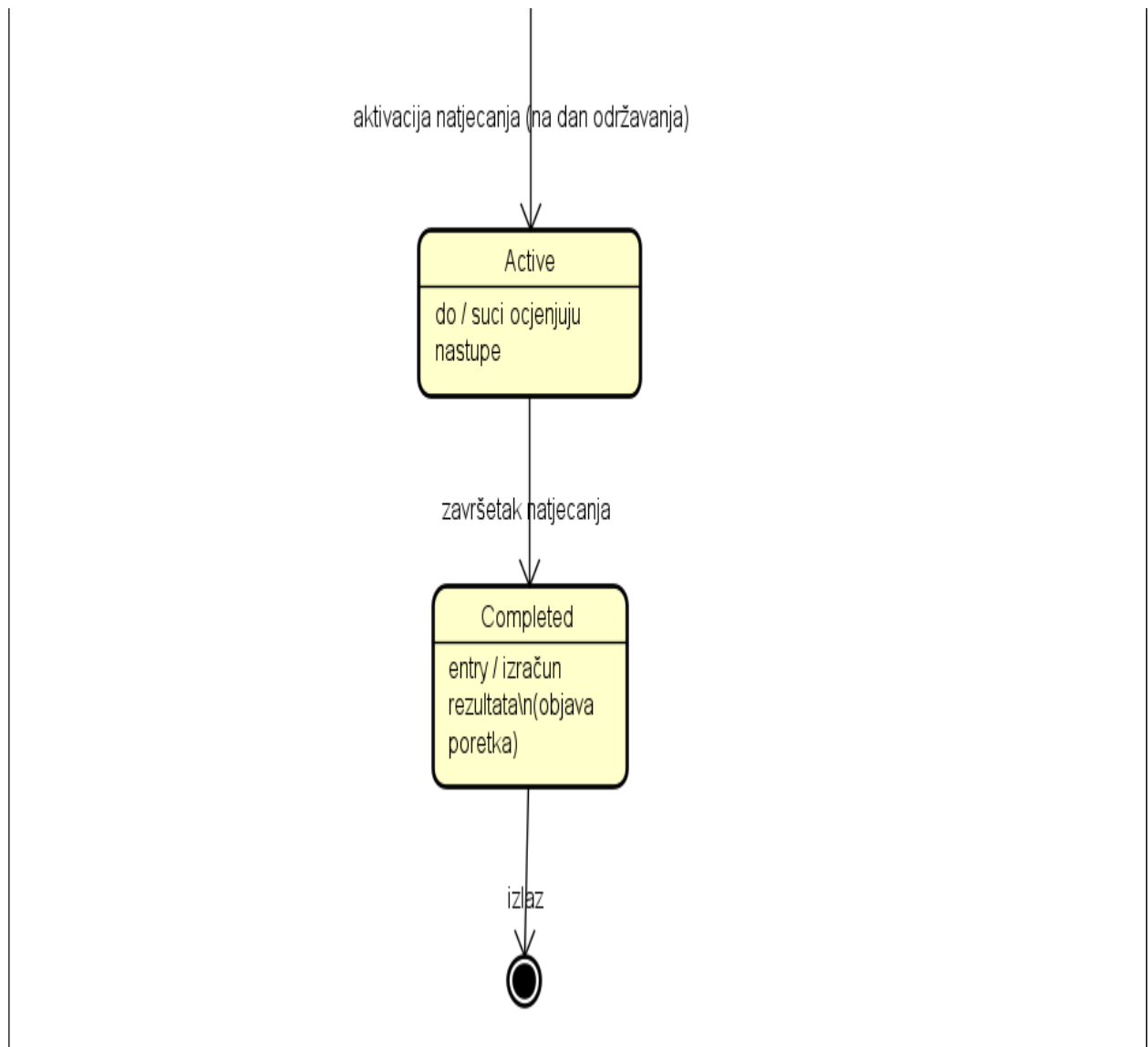




Dijagram stanja

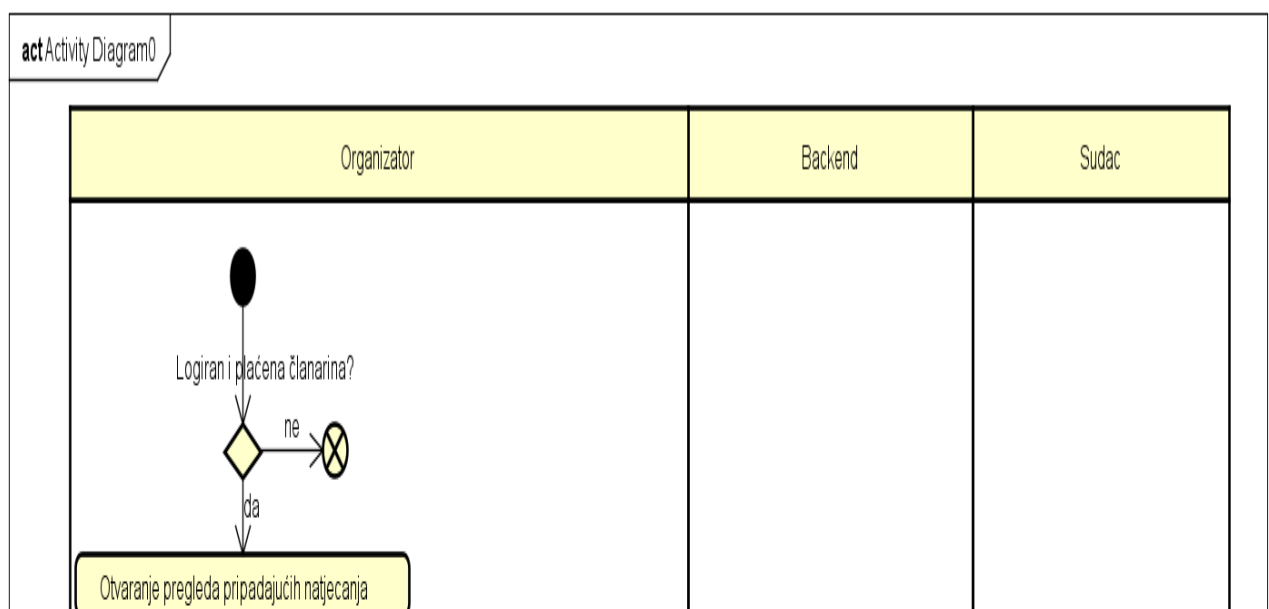
- Ovdje dijagram stanja prikazuje potpuni životni tijek natjecanja. Sve od otvaranja natjecanja do završetka natjecanja.

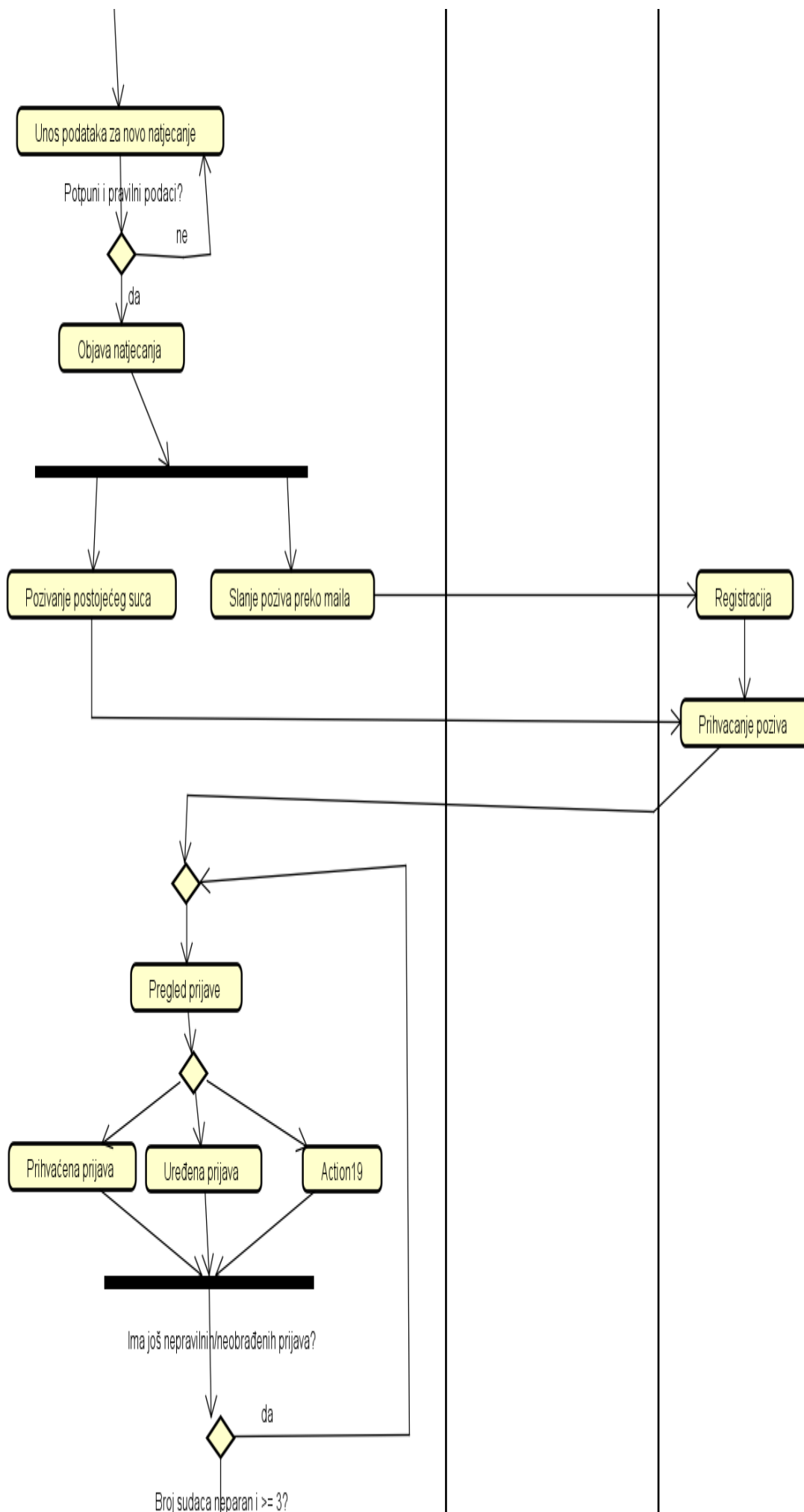


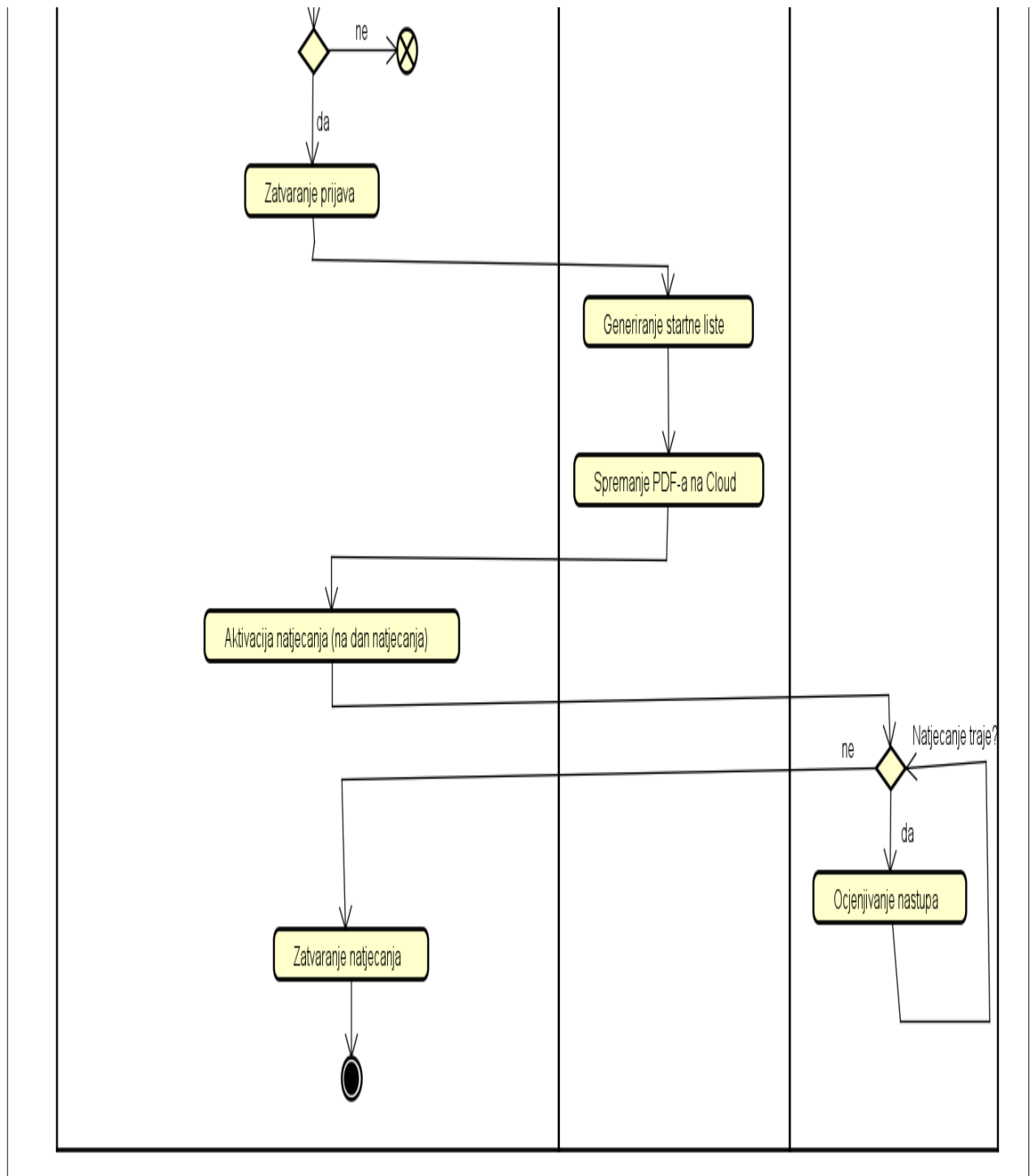


Dijagram aktivnosti

- Ovdje dijagram aktivnosti prikazuje potpuni životni tijek natjecanja iz perspektive organizatora i sudaca. Sve od otvaranja natjecanja do završetka natjecanja.

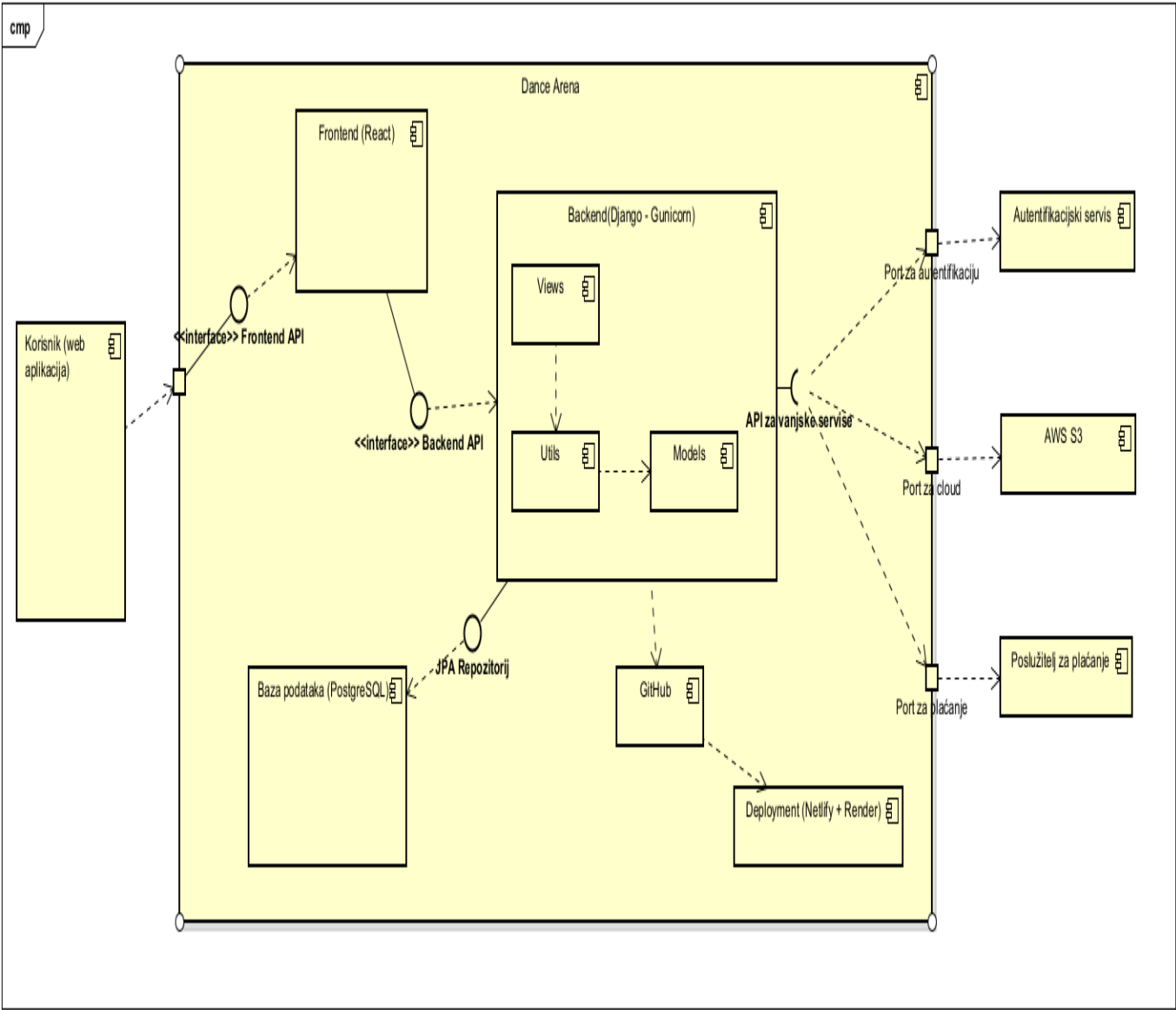






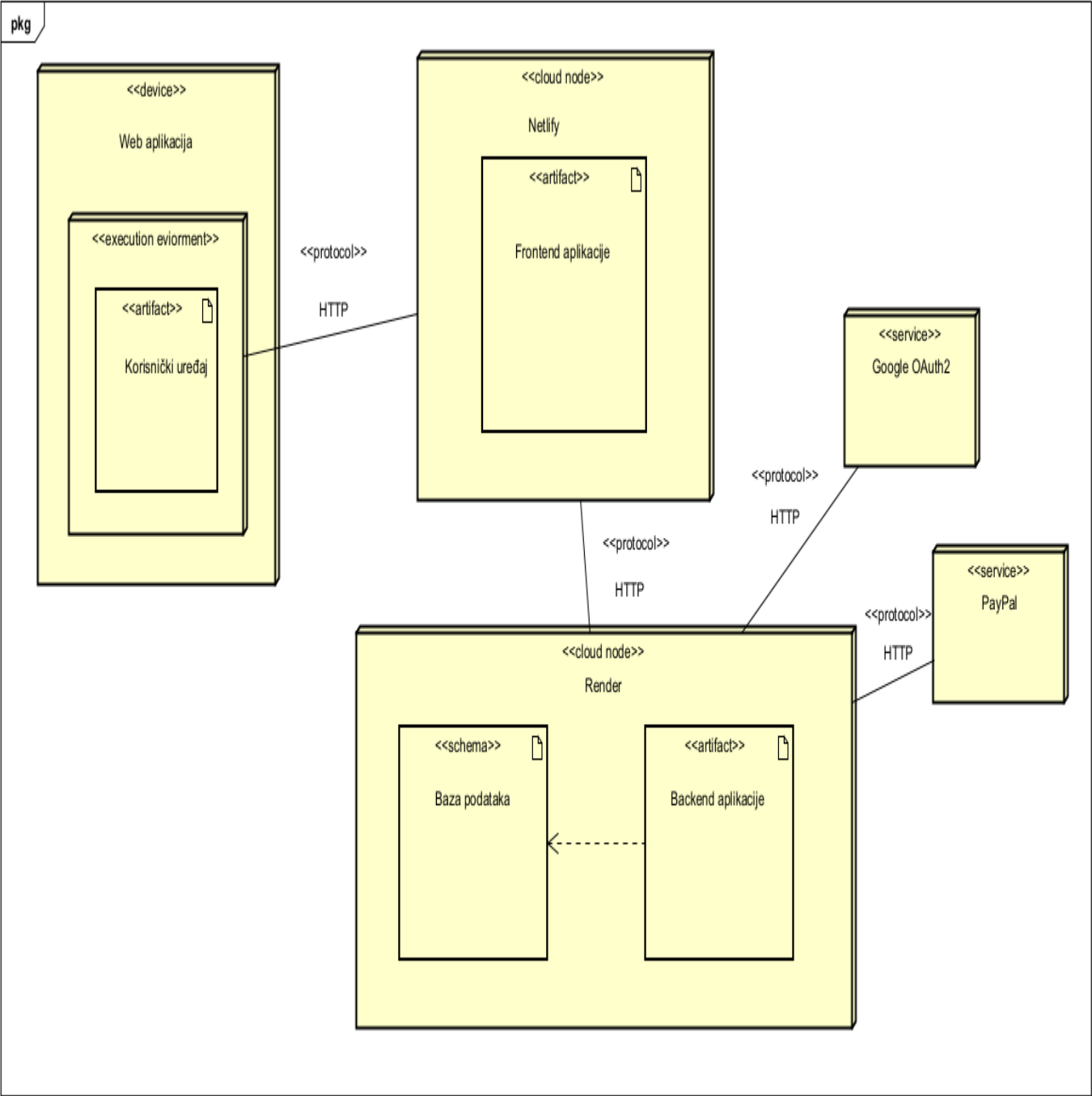
Dijagram komponenata

Dijagram prikazuje interakcije između dijelova aplikacije te interakcije aplikacije s korisnikom i vanjskim sustavima te njihove poveznice.



Dijagram razmještaja

Ovaj dijagram prikazuje interakcije i poveznice komponenta sustava te njihove poslužitelje u implementaciji.



Ispitivanje komponenti

Za ispitivanje komponenti korišten je Django API Test Case.

Testiranje komponenti odnosi se na provjeru ispravnosti pojedinačnih dijelova aplikacije. U našem slučaju testirane su funkcionalnosti povezane s informacijama korisničkih računa te općenite funkcionalnosti vezane uz natjecanja (stvaranje, uređivanje, objavljivanje, itd.).

Sljedeći su navedeni primjeri uspješno izvedenih testova u gore spomenutom programu.

```
test_edit_competition_forbidden_not_owner (competitions.tests.MyCompetitionsTests.test_edit_competition_forbidden_not_owner) ...
Testing test Edit Competition Forbidden Not Owner (Unauthorized user)
Test passed: System correctly blocked non-owner from editing.
ok
test_edit_competition_not_draft (competitions.tests.MyCompetitionsTests.test_edit_competition_not_draft) ...
Testing test Edit Competition Not Draft (Status check)
Test passed: System correctly blocked editing of a non-draft competition.
ok
test_edit_competition_success (competitions.tests.MyCompetitionsTests.test_edit_competition_success) ...
Testing test Edit Competition Success for ID: 1
Test passed: Successfully edited competition with ID: 1
Verified: New name is 'Izmijenjeno Ime Natjecanja'
ok
```

- Prikaz uspješno provedenog testova za uređivanje stvorenih natjecanja.

1. Ispis informacija o korisniku

U ovom primjeru testiramo prikaz dobavljanja informacija o pojedinom korisniku.

1.1. Autentificirani korisnik: redovni slučaj

U prvom testnom primjeru dobivljaju se informacije autentificiranog korisnika, stoga očekujemo na izlazu dobiti pripadne podatke korisnika u pitanju.

```
Test passed: Successfully retrieved data for user: testplesac

{"username": "testplesac",
 "email": "test@ples.com",
 "first_name": "Ivan",
 "last_name": "Ivić",
 "role": "ORGANIZER",
 "contact": "0993838383"}
ok
```

- Uspješan test 1.1.

1.2. Neautentificirani korisnik: nepostojeća funkcionalnost

U drugom testnom primjeru dobavljaju se informacije neautentificiranog korisnika, stoga očekujemo na izlazu dobiti obavijest kako se radi o nepostojećem korisniku.

```
test_current_user_not_authenticated (users.tests.CurrentUserTests.test_current_user_not_authenticated) ...
Testing testCurrentUserNotAuthenticated (Guest access)
Test passed: Successfully identified unauthenticated user
ok
```

- Uspješan test 1.2.

2. Dodjela neispravnih atributa korisniku

U ovom primjeru testiramo kako sustav reagira kada dodjeljujemo određene uloge korisničkom računu.

2.1. Uloga CLUB_MANAGER: redovni slučaj

U prvom testnom primjeru korisniku dodjeljujemo postojeću ulogu CLUB_MANAGER, stoga očekujemo da sustav dodijeli ulogu bez greške.

```
Testing test Update Club Manager Success with User ID: 1
Sent data: {'role': 'CLUB_MANAGER', 'name': 'Marko', 'surname': 'Marković', 'club_name': 'Plesni Studio X', 'club_location': 'Split'}
Test passed: Successfully updated user to CLUB_MANAGER with ID: 1

    "username": plesac123,
    "email": ,
    "first_name": Marko,
    "last_name": Marković,
    "role": CLUB_MANAGER,
    "club_name": Plesni Studio X,
    "club_location": Split
ok
```

- Uspješan test 2.1.

2.2. Nepostojeća uloga: nepostojeća funkcionalnost

U drugom primjeru korisniku dodjeljujemo nepostojeću ulogu, stoga očekujemo da ju sustav odbije dodijeliti korisniku.

```
test_update_invalid_role (users.tests.UserUpdateTests.test_update_invalid_role) ...
Testing test Update Invalid Role (Negative test)
Sent data: {'role': 'NEPOSTOJECA_ULOGA', 'name': 'Greška'}
Test passed: System correctly rejected invalid role
ok
```

- Uspješan test 2.2.

3. Dobavljanje natjecanja

U ovom primjeru testiramo dobavljanje natjecanja koje pripadaju / na koja je prijavljen određeni korisnik.

3.1. Sudac bez natjecanja: rubni slučaj

U prvom testnom primjeru korisnik čija natjecanja pokušavamo dobiti nema pripadna natjecanja, stoga očekujemo da sustav javi kako korisnik u pitanju nije dio niti jednog natjecanja.

```
test_my_competitions_empty (competitions.tests.MyCompetitionsTests.test_my_competitions_empty) ...
Testing test MyCompetitions Empty for User: sudac1
Test passed: Correctly identified that user 'sudac1' has no competitions.
ok
```

- Uspješan test 3.1.

3.2. Organizator s natjecanjima: redovni slučaj

U drugom testnom primjeru dobavljamo natjecanja korisnika koji ima pripadna natjecanja, stoga očekujemo da sustav ispiše sva njemu pripadna natjecanja.

```
test_my_competitions_success (competitions.tests.MyCompetitionsTests.test_my_competitions_success) ...
Testing test MyCompetitions Success for User: organizator1
Test passed: Successfully retrieved 3 competition(s) for organizer: organizator1
Competition(s) found: Dalmacija Dance Cup, Zagreb Open, Zadar Open
ok
```

- Uspješan test 3.2.

4. Otvaranje natjecanja

U ovom primjeru testiramo uspješnost otvaranja natjecanja ovisno o unesenim podacima.

4.1. Nepotpuni podaci: rubni slučaj

U prvom primjeru pokušavamo otvoriti natjecanje bez unošenja traženih podataka, stoga očekujemo da sustav ne dopusti stvaranje takvog natjecanja.

```
test_new_competition_incomplete (competitions.tests.MyCompetitionsTests.test_new_competition_incomplete) ...
Testing test New Competition incomplete for User: organizator1
Test passed: Correctly denied creation to user with id: 1
                sent competition information:
                {'name': 'Zabranjeno natjecanje'}
ok
```

- Uspješan test 4.1.

4.2. Potpuni podaci: redovni slučaj

U drugom primjeru pokušavamo otvoriti natjecanje nakon unošenja svih potrebnih podataka, stoga očekujemo da sustav dozvoli otvaranje takvog natjecanja.

```
test_new_competition_success (competitions.tests.MyCompetitionsTests.test_new_competition_success) ...
Testing test New Competition Success for User: organizator1
Test passed: Successfully created competition with ID: 3
ok
```

- Uspješan test 4.2.

5. Uređivanje natjecanja

U ovom primjeru testiramo sve moguće slučajeve pri uređivanju natjecanja.

5.1. Korisnik bez natjecanja pokušava uređivati: nepostojeća funkcionalnost

U prvom primjeru testiramo uređivanje natjecanja gdje korisnik koji uređuje natjecanje ga nije ujedno i otvorio, stoga očekujemo da sustav zabrani ovaj pokušaj uređivanja.

```
Testing test Edit Competition Forbidden Not Owner (Unauthorized user)
Test passed: System correctly blocked non-owner from editing.
ok
```

- Uspješan test 5.1.

5.2. Non-draft natjecanja: rubni slučaj

U drugom primjeru testiramo uređivanje natjecanja koje nije draft, stoga očekujemo da sustav zabrani ovakvo uređivanje natjecanja.

```
test_edit_competition_not_draft (competitions.tests.MyCompetitionsTests.test_edit_competition_not_draft) ...
Testing test Edit Competition Not Draft (Status check)
Test passed: System correctly blocked editing of a non-draft competition.
ok
```

- Uspješan test 5.2.

5.3. Draft natjecanje: redovni slučaj

U trećem primjeru testiramo uređivanje natjecanja koje je draft, stoga očekujemo da sustav dozvoli ovakvo uređivanje.

```
test_edit_competition_success (competitions.tests.MyCompetitionsTests.test_edit_competition_success) ...
Testing test Edit Competition Success for ID: 1
Test passed: Successfully edited competition with ID: 1
Verified: New name is 'Izmijenjeno Ime Natjecanja'
ok
```

- Uspješan test 5.3.

6. Objavljivanje natjecanja

U ovom primjeru testiramo uspješnost objavljivanja natjecanja ovisno o statusu vlasništva korisnika i trenutnom statusu natjecanja.

6.1. Već aktivno natjecanje: rubni slučaj

U prvom testnom primjeru testiramo objavljivanje natjecanja koje je već počelo, stoga očekujemo da sustav zabrani objavljivanje ovakvog natjecanja.

```
Testing test Publish Non Draft (Double publish check)
Test passed: System correctly blocked re-publishing an already active competition.
ok
```

- Uspješan test 6.1.

6.2. Korisnik koji ne posjeduje natjecanje: nepostojeća funkcionalnost

U drugom testnom primjeru testiramo objavljivanje natjecanja od strane korisnika koji ne posjeduje to isto natjecanje, stoga očekujemo da sustav zabrani objavljivanje ovakvog natjecanja.

```
test_publish_competition_not_owner (competitions.tests.MyCompetitionsTests.test_publish_competition_not_owner) ...
Testing test Publish Competition Not Owner (Unauthorized attempt)
Test passed: Access denied for non-owner.
ok
```

- Uspješan test 6.2.

6.3. Ispravno natjecanje i korisnik: redovni slučaj

U trećem testnom primjeru testiramo objavljivanje natjecanja koje nije ranije objavljeno od strane korisnika kojemu to isto natjecanje pripada, stoga očekujemo da sustav dozvoli objavljivanje ovakvog natjecanja.

```
test_publish_competition_success (competitions.tests.MyCompetitionsTests.test_publish_competition_success) ...
Testing test Publish Competition Success for ID: 1
Test passed: Competition 'Dalmacija Dance Cup' is now PUBLISHED.
Verified status in database: PUBLISHED
ok
```

- Uspješan test 6.3.

Ispitivanje sustava

Za testiranje sustava korišten je Playwright, koji omogućuje automatizirano testiranje stvarnih korisničkih scenarija kroz web preglednik. Pomoću Playwrighta testirane su ključne funkcionalnosti sustava poput prijave korisnika, stvaranja natjecanja, itd. Korištenjem Playwrighta osigurana je ispravnost integracije između

frontend i backend dijela aplikacije te stabilnost i konzistentno ponašanje korisničkog sučelja na različitim preglednicima.

1. Funkcionalnosti neprijavljenog korisnika: redovni slučaj

U ovom primjeru testiramo mogućnost neregistriranog korisnika da pregleda trenutna natjecanja, rezultate gotovih natjecanja te na kraju navigira na login page. Očekujemo da sustav dozvoli sve navedene akcije neregistriranom korisniku.

Test Steps	
> ✓ Before Hooks	1.6s
> ✓ Navigate to "/homepage" — Anon.spec.js:4	783ms
> ✓ Click getByText('Rezultati') — Anon.spec.js:5	1.4s
> ✓ Click getByText('Natjecanja') — Anon.spec.js:6	414ms
> ✓ Click locator('i').nth(2) — Anon.spec.js:7	321ms
> ✓ Click locator('i').nth(2) — Anon.spec.js:8	141ms
> ✓ Click locator('div').nth(4) — Anon.spec.js:9	162ms
> ✓ After Hooks	987ms

- Uspješan test 1

2. Otvaranje natjecanja kao organizator: redovni slučaj

U ovom primjeru testiramo mogućnost organizatora da otvori natjecanje. Očekujemo da sustav dozvoli organizatoru otvaranje natjecanja povodom unosa svih traženih podataka.

Test Steps	
> ✓ Before Hooks	1.5s
> ✓ Navigate to "/homepage" — Org.spec.js:8	1.9s
> ✓ Click getByRole('img', { name: 'User Icon' }) — Org.spec.js:9	1.1s
> ✓ Click getByRole('link', { name: 'Profile' }) — Org.spec.js:10	120ms
> ✓ Click getByRole('button', { name: 'Novo Natjecanje' }) — Org.spec.js:11	100ms
> ✓ Click getByRole('textbox', { name: 'Naziv natjecanja' }) — Org.spec.js:12	211ms
> ✓ Fill "test" getByRole('textbox', { name: 'Naziv natjecanja' }) — Org.spec.js:13	31ms
> ✓ Fill "2026-01-20" getByRole('textbox', { name: 'Početak natjecanja' }) — Org.spec.js:14	46ms
> ✓ Click getByRole('textbox', { name: 'Lokacija natjecanja' }) — Org.spec.js:15	138ms
> ✓ Fill "test" getByRole('textbox', { name: 'Lokacija natjecanja' }) — Org.spec.js:16	37ms
> ✓ Click getByRole('textbox', { name: 'Opis natjecanja' }) — Org.spec.js:17	51ms
> ✓ Fill "test" getByRole('textbox', { name: 'Opis natjecanja' }) — Org.spec.js:18	36ms
> ✓ Click locator('svg').first() — Org.spec.js:19	68ms
> ✓ Click getByRole('option', { name: 'Breakdance' }) — Org.spec.js:20	154ms
> ✓ Click locator('svg').nth(3) — Org.spec.js:21	105ms
> ✓ Click getByRole('option', { name: 'Juniori' }) — Org.spec.js:22	76ms
> ✓ Click locator('.select_value-container.select_value-container--is-multi.css-hlgwow > .select_input-container') — Org.spec.js:23	82ms
> ✓ Click getByRole('option', { name: 'Duo' }) — Org.spec.js:24	92ms
> ✓ Click getByRole('spinbutton', { name: 'Iznos kotizacije (€):' }) — Org.spec.js:25	100ms
> ✓ Fill "10" getByRole('spinbutton', { name: 'Iznos kotizacije (€):' }) — Org.spec.js:26	48ms
> ✓ Click getByRole('button', { name: 'Kreiraj natjecanje' }) — Org.spec.js:31	105ms
> ✓ After Hooks	619ms

- Uspješan test 2

3. Otvaranje natjecanja bez potrebnih podataka: rubni slučaj

U ovom primjeru testiramo mogućnost otvaranja natjecanja bez potrebnih podataka. Očekujemo da sustav ne dozvoli otvaranje natjecanja pod tim uvjetima.

Q	All 1	✓ Passed 1	Failed 0	Flaky 0	Skipped 0	🕒	⚙️
---	-------	------------	----------	---------	-----------	---	----

test

example.spec.js:7

 View Trace | 10.4s

✓ Run

▼ Test Steps

- > ✓ Before Hooks 2.2s
- > ✓ Navigate to "/homepage" — example.spec.js:8 1.3s
- > ✓ Click getByRole('link', { name: 'Novo natjecanje' }) — example.spec.js:9 558ms
- > ✓ Click getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:10 737ms
- > ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:11 67ms
- > ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:12 52ms
- > ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:13 78ms
- > ✓ Fill "T" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:14 126ms

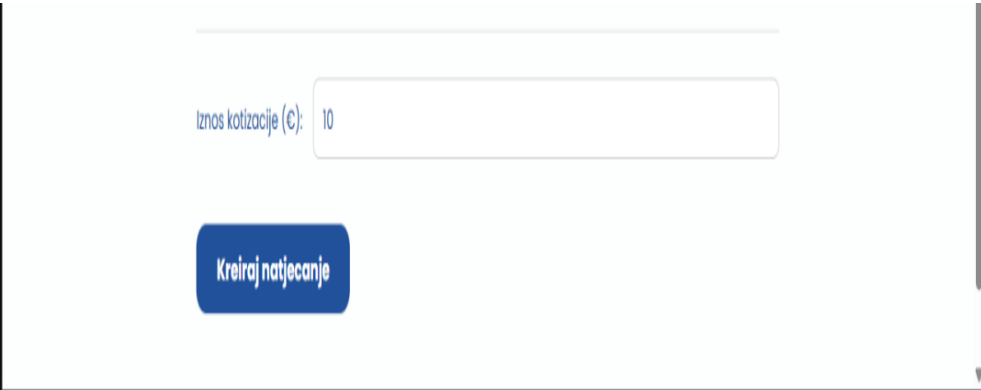
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:15	242ms
> ✓ Fill "Test" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:16	121ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:17	71ms
> ✓ Fill "TestB" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:18	128ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:19	41ms
> ✓ Fill "TestBez" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:20	125ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:21	46ms
> ✓ Fill "TestBezO" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:22	139ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:23	50ms
> ✓ Fill "TestBez" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:24	122ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:25	55ms
> ✓ Fill "TestBezL" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:26	102ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:27	46ms

> ✓ Fill "TestBezLokacije" getByRole('textbox', { name: 'Naziv natjecanja:' }) — example.spec.js:28	96ms
> ✓ Fill "2026-01-30" getByRole('textbox', { name: 'Početak natjecanja:' }) — example.spec.js:29	86ms
> ✓ Click getByRole('textbox', { name: 'Lokacija natjecanja:' }) — example.spec.js:30	111ms
> ✓ Click getByRole('textbox', { name: 'Opis natjecanja:' }) — example.spec.js:31	101ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Opis natjecanja:' }) — example.spec.js:32	81ms
> ✓ Fill "T" getByRole('textbox', { name: 'Opis natjecanja:' }) — example.spec.js:33	88ms
> ✓ Press "CapsLock" getByRole('textbox', { name: 'Opis natjecanja:' }) — example.spec.js:34	40ms
> ✓ Fill "Test" getByRole('textbox', { name: 'Opis natjecanja:' }) — example.spec.js:35	88ms
> ✓ Click locator('.select_input-container').first() — example.spec.js:36	183ms
> ✓ Click getByRole('option', { name: 'Breakdance' }) — example.spec.js:37	230ms
> ✓ Click locator('div:nth-child(3) > .basic-multi-select > .select_control > .select_value-container > .select_...') — example.spec.js:38	305ms
> ✓ Click getByRole('option', { name: 'Solo' }) — example.spec.js:39	225ms
> ✓ Click locator('.select_value-container.select_value-container--is-multi.css-hlgwow > .select_input-cont...') — example.spec.js:40	153ms
> ✓ Click getByRole('option', { name: 'Juniori' }) — example.spec.js:41	198ms

- > ✓ Click `getByRole('spinbutton', { name: 'Iznos kotizacije (€):' })` — `example.spec.js:42` 189ms
- > ✓ Fill "10" `getByRole('spinbutton', { name: 'Iznos kotizacije (€):' })` — `example.spec.js:43` 108ms
- > ✓ Click `getByRole('button', { name: 'Kreiraj natjecanje' })` — `example.spec.js:44` 146ms
- > ✓ After Hooks 2.2s

▼ Screenshots

The screenshot shows a web form for 'DANCE ARENA'. At the top, there are links for 'Novo natjecanje' and 'Moja natjecanja', and a user profile for 'Marko'. The form has several sections: a text input for 'Unesite lokaciju natjecanja', a text area for 'Opis natjecanja:' with a placeholder 'Test' and a validation error 'Please fill out this field.', two dropdown menus for 'Stilovi:' (currently 'Breakdance') and 'Dobne kategorije:' (currently 'Juniori'), and a dropdown for 'Veličine grupa:' (currently 'Solo'). Each dropdown has a close button (X) and a dropdown arrow (v).



 [screenshot](#)

Traces



 [trace](#)

- Uspješan test 3

4. Korištenje nepostojeće funkcije: nepostojeća funkcionalnost

U ovom primjeru vrlo direktno testiramo responzivnost sustava na nepostojeće funkcije. Očekujemo da sustav javi grešku i ne izvrši nepostojeću funkciju.

▼ Errors

Error: Test je pao jer su se pojavile JS greške: window.nePostojecaFunkcija is not a function

Copy prompt

expect(received).toHaveLength(expected)

Expected length: 0

Received length: 1

Received array: ["window.nePostojecaFunkcija is not a function"]

```
26 | // Provjeravamo da je lista grešaka prazna.
27 | // Budući da će 'nePostojecaFunkcija' napuniti listu, ovaj expect će FAIL-ati.
> 28 | expect(errors, `Test je pao jer su se pojavile JS greške: ${errors.join(', ')}').toHaveLength(0);
    |                                     ^
29 | });
    at C:\Users\Luka\Desktop\DANCEARENA\DanceArena\Source Code\frontend\tests\example.spec.js:28:84
```

▼ Test Steps

> ✓ Before Hooks	2.8s
> ✓ Navigate to "/homepage" — example.spec.js:17	1.5s
> ✓ Click getByText('Rezultati') — example.spec.js:20	857ms
> ✓ Wait for timeout — example.spec.js:23	1.0s
> ✗ Test je pao jer su se pojavile JS greške: window.nePostojecaFunkcija is not a function — example.spec.js:28	8ms
> ✓ After Hooks	606ms
> ✓ Worker Cleanup	933ms

- Uspješan test 4

5. Mijenjanje cijena pretplate kao administrator: redovni slučaj

U ovom primjeru testiramo logiranje administratora i mijenjanje cijene pretplate. Očekujemo da sustav ovo dozvoli jer je to standardna funkcionalnost za administratore.

✓ Test Steps	
> ✓ Before Hooks	2.2s
> ✓ Navigate to "/admin/login/?next=/admin/" — example.spec.js:8	508ms
> ✓ Click getByRole('textbox', { name: 'Username:' }) — example.spec.js:9	178ms
> ✓ Fill "admin" getByRole('textbox', { name: 'Username:' }) — example.spec.js:10	69ms
> ✓ Click getByRole('textbox', { name: 'Password:' }) — example.spec.js:11	83ms
> ✓ Fill "123" getByRole('textbox', { name: 'Password:' }) — example.spec.js:12	34ms
> ✓ Click getByRole('button', { name: 'Log in' }) — example.spec.js:13	1.5s
> ✓ Click getByRole('link', { name: 'Organizer subscription prices' }) — example.spec.js:14	280ms
> ✓ Click getByRole('link', { name: 'Add organizer subscription' }) — example.spec.js:15	423ms
> ✓ Click getByRole('spinbutton', { name: 'Price:' }) — example.spec.js:16	112ms
> ✓ Fill "15" getByRole('spinbutton', { name: 'Price:' }) — example.spec.js:17	33ms
> ✓ Click getByRole('button', { name: 'Save', exact: true }) — example.spec.js:18	2.8s
> ✓ After Hooks	 485ms

- Uspješan test 5

Korištene tehnologije i alati

Programski jezici

Za razvoj poslužiteljskog dijela aplikacije korišten je **Python (verzija 3.12)**, objektno orijentirani programski jezik poznat po jednostavnoj sintaksi i velikom broju dostupnih biblioteka, što omogućuje brzi razvoj i čitljiv kod.

Klijentski dio aplikacije razvijen je korištenjem **JavaScript (ES2022 standard)**, koji predstavlja standardni jezik za razvoj modernih web aplikacija i omogućuje izradu interaktivnih korisničkih sučelja.

Radni okviri i biblioteke

Backend aplikacije implementiran je pomoću **Django radnog okvira (verzija 5.0)**. Django omogućuje brzu izradu web aplikacija kroz ugrađene mehanizme za rad s bazom podataka, autentifikaciju korisnika i administracijsko sučelje, čime se smanjuje količina potrebnog ručnog koda i mogućnost pogrešaka.

Frontend aplikacije razvijen je korištenjem **React biblioteke (verzija 19.1.1)**. React omogućuje razvoj korisničkog sučelja temeljenog na komponentama, što olakšava ponovnu uporabu koda i održavanje aplikacije.

Za razvoj i pokretanje frontend dijela korišten je **Vite (verzija 7.1.7)**, alat koji omogućuje brzo pokretanje razvojnog okruženja i trenutno osvježavanje promjena bez ponovnog učitavanja cijele stranice.

Za dizajn korisničkog sučelja i izradu prototipa korišten je alat **Figma**, koji omogućuje kolaborativan rad tima na dizajnu sučelja.

Baza podataka

Za pohranu podataka korištena je relacijska baza podataka **PostgreSQL (verzija 18.0)**. PostgreSQL pruža visoku pouzdanost, podršku za složene upite i integritet podataka, što je ključno za sustav koji upravlja korisnicima, natjecanjima i financijskim transakcijama.

Razvojni alati

Kao razvojno okruženje korišten je **Visual Studio Code**, lagani i proširivi uređivač koda koji nudi podršku za Python, JavaScript i rad s Git repozitorijima.

Za upravljanje izvornim kodom i verzijama korišten je **Git (verzija 2.52)**. Git omogućuje praćenje promjena u kodu, timski rad te jednostavno vraćanje na prethodne verzije sustava.

Alati za ispitivanje

Za ispitivanje funkcionalnosti sustava korišten je Django API TestCase, koji omogućuje provedbu jediničnih i integracijskih testova nad REST API slojem aplikacije. Ovim alatom testirane su ključne komponente sustava čime se osigurava ispravnost poslovne logike i stabilnost aplikacije.

Za testiranje sustava korišten je Playwright, koji omogućuje automatizirano testiranje korisničkih scenarija kroz web preglednik te provjeru ispravnosti rada ključnih funkcionalnosti i integracije između frontend i backend dijela aplikacije.

Cloud platforma

Frontend dio aplikacije hostan je na platformi Netlify, koja omogućuje jednostavno razmještanje statičkih web aplikacija, automatsko ponovno izgrađivanje aplikacije prilikom promjena u repozitoriju te HTTPS podršku.

Backend dio aplikacije, zajedno s bazom podataka, hostan je na platformi Render, koja omogućuje pouzdano izvođenje Django aplikacije, upravljanje okruženjem te skalabilno i sigurno hostanje PostgreSQL baze podataka.

1. Preduvjeti

Prije svega instalirati:

- Git
- Python 3 (preporuka 3.11 ili noviji)
- PostgreSQL (lokalno)
- Node.js i npm

2. Kloniranje repozitorija

1. Otvori terminal (PowerShell ili CMD).
2. Pokreni:

```
git clone https://github.com/TheStickOfTruths/DanceArena.git
```

```
cd DanceArena
```

3. Backend – Django + PostgreSQL

3.1. Ulazak u backend direktorij

```
cd "Source Code/backend" (gdje se nalazi manage.py)
```

3.2. Kreiranje i aktivacija virtualnog okruženja (venv)

U PowerShellu:

```
python -m venv venv_be
```

```
venv_be\Scripts\Activate.ps1 (<Windows>) (Linux/Mac>) source venv_be/bin/activate
```

Ako je sve u redu, na početku linije u terminalu pojavit će se (venv).

3.3. Instalacija Python paketa

U PowerShellu:

```
pip install -r requirements.txt
```

3.4. Stvaranje korisnika koji može pristupiti bazi

Jednokratno je potrebno napraviti korisnika i bazu:

- Baza: DanceArenaLocal
- Korisnik: DanceArenaUser
- Lozinka: dancearena

Otvoriti Command Prompt i ukucati sljedeću naredbu:

```
psql -U postgres
```

Zatim ukucati lozinku za PGAdmin. Kada se pojavi postgres=# spojeni ste na PostgreSQL i možete izvršavati SQL naredbe.

Nakon spajanja izvršite sljedeće naredbe da kreirate bazu:

```
CREATE ROLE "DanceArenaUser" LOGIN PASSWORD 'dancearena';
```

```
CREATE DATABASE "DanceArenaLocal" OWNER "DanceArenaUser";
```

Na kraju, za izlaz iz psql a ukucajte naredbu: \q

3.6 Preuzimanje i kopiranje .env datoteka

1. Preuzmi .env datoteke s Teamsa (backend i frontend).
2. Kopiraj ih u odgovarajuće foldere u projektu:
 - Backend .env → u folder gdje je manage.py (Source Code/backend/.env)
 - Frontend .env → u folder gdje je package.json (Source Code/frontend/.env)

3.7. Migracije i pokretanje backend servera

U aktiviranom venv u:

```
python manage.py migrate
```

```
python manage.py createsuperuser (opcionalno)
```

```
python manage.py runserver
```

Backend bi sada trebao raditi na: <http://127.0.0.1:8000/>

4. Frontend – React

4.1. Ulazak u frontend direktorij

U novom terminal tabu ili prozoru:

```
cd "Source Code/frontend"
```

(tu se nalazi package.json)

4.2. Kreiranje i aktivacija virtualnog okruženja (venv)

U PowerShellu:

```
deactivate (samo ako ste u istom terminalu kako biste isključili okruženje backenda)
```

```
python -m venv venv_fe
```

```
venv_fe\Scripts\Activate.ps1 (<Windows>) (Linux/Mac>) source venv_fe/bin/activate
```

Ako je sve u redu, na početku linije u terminalu pojavit će se (venv).

4.3. Instalacija JS ovisnosti

U terminalu pokrenuti:

```
npm install
```

4.4. Pokretanje frontend dev servera

U terminalu pokrenuti:

```
npm start
```

Frontend će standardno biti na: <http://localhost:5173> (ili port koji ispiše terminal).

5. Sažetak koraka

Backend komande

```
cd "Source Code/backend"
```

```
python -m venv venv_be
```

```
venv_be\Scripts\Activate.ps1
```

```
pip install -r requirements.txt
```

Kreirati lokalnu Postgres bazu DanceArenaLocal (vidi gore)

```
python manage.py migrate
```

```
python manage.py runserver
```

Frontend komande

```
cd "Source Code/frontend"
```

```
python -m venv venv_fe
```

```
venv_fe\Scripts\Activate.ps1
```

```
npm install
```

```
npm run dev
```

Izrada web aplikacije Dance Arena je bio zahtjevan zadatak, ali uz dobru raspodjelu posla i vremena te učestale sastanke i dogovore za sljedeće korake razvoj se odvijao poprilično glatko.

Tim od šestero smo odmah razdijelili na 3 para; backend, frontend i dokumentaciju. Komunikacija se odvijala učestalim sastancima tijekom kojih bi se dogovarali oko većih odluka aplikacije, te preko zajedničke grupe za manje stvari. Također, dijeljenje i implementacija novog koda se odvijala preko zajedničkog GitHub projekta.

U prvoj fazi smo, uz raspodjelu uloga, odredili programske jezike i aplikacije koje ćemo koristiti u razvoju, napravili sve potrebne grupa za komunikaciju, definirali funkcijske i nefunkcijske zahtjeve aplikacije te odlučili koje ćemo vanjske servise koristiti u aplikaciji.

Druga faza se sastojala od postavljanja temelja za aplikaciju poput prototipa stranice, spajanja API za log in, uspostavljanja baze podataka i slično. U dokumentaciji smo tad zapisali prijašnje osmišljene zahtjeve sustava, obrasce uporabe i zapisali specifikacije baze podataka. Kraj ciklusa je obilježio neočekivano težak, ali uspješan deploy te prikaz funkcionalnog log in sustava

Nakon pauze za praznike smo započeli treću fazu koja se bavila završavanjem svega nedovršenog u prvom ciklusu, sad uz već postavljene temelje. Sastanci uživo su postali rjeđi jer je plan bio izvrsno definiran unaprijed te je svatko znao što treba dovršiti. Pred kraj se posao morao malo ubrzati kako bi se sve stiglo, ali smo na kraju sve funkcionalnosti uspješno osmislili, implementirali i deployali.

Unatoč težini i količini posla kojeg je razvoj Dance Arene pružao, mislimo da je projekt dobro obavljen. Naša najveća snaga je definitivno bila odlična organizacija, komunikacija i raspodjela posla, što je maksimiziralo učinkovitost vremena uloženog u projekt. Tim je bio složan i kolegijalan te su svi bili voljni pomoći kad je zatrebalo.

Ako bi se projekt nastavio i realizirao u pravu komercijalnu aplikaciju, nadogradnje bi uključivale izradu mobilne verzija aplikacije, mogućnost promjene jezika, funkcionalnosti dopisivanja i razne stavke pristupačnosti.

Popis literature

1. **Programsko inženjerstvo** FER ZEMRIS. Dostupno na: <http://www.fer.hr/predmet/proinz>
 2. **Sommerville, I.** *Software Engineering*, 8th Edition. Addison-Wesley, 2007.
 3. **Lethbridge, T. C., Laganière, R.** *Object-Oriented Software Engineering*, 2nd Edition. McGraw-Hill, 2005.
 4. **Astah Community** *Astah UML – Community Edition*. Dostupno na: <http://astah.net/editions/uml-new>
 5. **Django Software Foundation** *Django Documentation*. Dostupno na: <https://docs.djangoproject.com/>
 6. **Python Software Foundation** *Python Documentation*. Dostupno na: <https://docs.python.org/>
 7. **Meta Platforms, Inc.** *React – Official Documentation*. Dostupno na: <https://react.dev/>
 8. **Vite Team** *Vite Documentation*. Dostupno na: <https://vitejs.dev/>
 9. **Figma, Inc.** *Figma Documentation*. Dostupno na: <https://help.figma.com/>
 10. **Hardt, D.** *The OAuth 2.0 Authorization Framework*. RFC 6749, Internet Engineering Task Force (IETF), 2012.
 11. **Google LLC** *Google Identity – OAuth 2.0*. Dostupno na: <https://developers.google.com/identity>
 12. **PayPal, Inc.** *PayPal Developer Documentation*. Dostupno na: <https://developer.paypal.com/>
-

Rev.	Opis promjene/dodatka	Autori	Datum
0.1	Napravljen predložak	Luka Rako	19.10.2025.
0.2	Većinski dovršena 1. stranica	Luka Rako	20.10.2025.
0.3	Dovršena 2. stranica	Luka Rako, Kristijan Karanušić	21.10.2025.
0.4	Dovršena 3. stranica osim opisa osnovnog tijeka	Luka Rako	5.11.2025.
0.5	Dovršena 4. stranica	Kristijan Karanušić	13.11.2025.
0.6	Dovršena 5. stranica	Luka Rako	11.1.2026.
0.7	Stvoren popis literature	Kristijan Karanušić	13.01.2026.
0.8	Većinski dovršena 7. stranica	Kristijan Karanušić	13.01.2026.
0.9	Testiranje komponenti dodano 6. stranici.	Kristijan Karanušić	20.01.2026.
0.10	Dovršena 9. stranica	Luka Rako	22.01.2026.
0.11	Dovršena 6. stranica	Kristijan Karanušić	23.01.2026.
0.12	Dovršena 8. stranica	Luka Rako	23.01.2026.

Evidencija promjena sadrži popis promjena izvršenih tijekom cijelo životnog trajanja predmeta evidencije (dokumentacije, projekta i sl.). Osnova svrha je praćenja napretka svake promjene na temelju njezina preispitivanja, odobrenja (ili odbijanja), provedbe, kao i zaključenja. Štoviše, dobar dnevnik promjena sadrži i datum promjene i njegov utjecaj na projekt u smislu rizika, vremena i troškova. Sve te promjene prenose se dionicima. Štoviše, odbačene promjene također su uključene u povijest promjena.

Zašto? Olakšano praćenje ključnim dionicima.

Zapisnici sa sastanaka

PRVI SASTANAK

Datum: 14.10.2025. **Trajanje:** 90 minuta

Prisutni:

- Kristijan Karanušić
- Roko Čačija
- Luka Golub
- Šimun Vrsalović
- Luka Rako
- Karlo Višnjovski

Teme:

1. Upoznavanje sa GitHub-om
2. Podjela poslova
3. Sažetak zadatka

DRUGI SASTANAK

Datum: 21.10.2025. **Trajanje:** 90 minuta

Prisutni:

- Kristijan Karanušić
- Roko Čačija
- Luka Golub
- Šimun Vrsalović
- Luka Rako
- Karlo Višnjovski

Teme:

1. Definiranje funkcijskih i nefunkcijskih zahtjeva projekta

TREĆI SASTANAK

Datum: 06.11.2025. **Trajanje:** 60 minuta

Prisutni:

- Kristijan Karanušić
- Roko Čačija
- Luka Golub
- Luka Rako
- Karlo Višnjovski

Teme:

- 1. Implementacija baze podataka
- 2. Diskutiranje obrazaca uporabe (Use Cases)
- 3. Plan pripreme za predaju

ČETVRTI SASTANAK

Datum: 09.01.2026. Trajanje: 45 minuta

Prisutni:

- Kristijan Karanušić
- Roko Čačija
- Luka Golub
- Luka Rako
- Karlo Višnjovski

Teme:

- 1. Utvrđivanje preostalih funkcionalnosti za Alpha verziju
- 2. Dogovor o načinu izrade preostale dokumentacije

Tablica aktivnosti

Roko Čačija

#	Aktivnost	Sati
1	Stvaranje modela u Django	3
2	Stvaranje strukture platforme pomoću URL-ova	1
3	Kodiranje funkcionalnih zahtjeva za natjecanja	8
4	Implementacija sudaca na platformi	1.5
5	Implementacija voditelja klubova na platformi	2
6	Implementacija organizatora na platformi	7
7	Ispunjavanje dokumentacije	0.5
8	Deployment aplikacije	11.5
9	Proučavanje materijala JWT	3
10	Implementacija JWT	3
11	Spremanje datoteka u cloud	4
12	Backend funkcije za korisnike	1

#	Aktivnost	Sati
13	Backend funkcije za natjecanja	3.5
14	Testiranje funkcija	2
15	Dokumentacija	1

Šimun Vrsalović

#	Aktivnost	Sati
1	Postavljanje Figma projekta	0.5
2	Dizajniranje korisničkog sučelja unutar Figma	3.5
3	Izgradnja korisničkog sučelja u Reactu	5.5
4	Spajanje Google OAuth sustava	5
5	Deployment aplikacije	12
6	Postavljanje i uređivanje GitHub repozitorija	2
7	Organizator funkcionalnosti	17
8	Voditelj funkcionalnosti	13
9	Sudac funkcionalnosti	8
10	Uređivanje stranice	12

Karlo Višnjovski

#	Aktivnost	Sati
1	Dizajniranje korisničkog sučelja unutar Figma	9
2	Izgradnja korisničkog sučelja u Reactu (HTML i CSS)	20
3	Implementacija responzivnog sučelja, opcija kontrasta i opcija čitljivosti	5
4	Izrada podataka za demonstraciju aplikacije	2

Luka Golub

#	Aktivnost	Sati
1	Postavljanje OAuth-a u backendu	10
2	Dodavanje osnovnih modela za registraciju	9
3	Postavljanje permissions-a na URL-ove	3
4	Postavljanje osnovnih dijelova Django servera	5
5	Izrada dokumentacije (odstupanja za UC-ove)	1

#	Aktivnost	Sati
6	Postavljanje DanceArena PayPal računa i implementacija pretplate	11.5
7	Implementacija plaćanja prijave za voditelje kluba	9.5
8	Testiranje stranice (inkl. workaround za Google OAuth)	12
9	Ispitivanje i stvaranje uputa za lokalno pokretanje aplikacije	1

Luka Rako

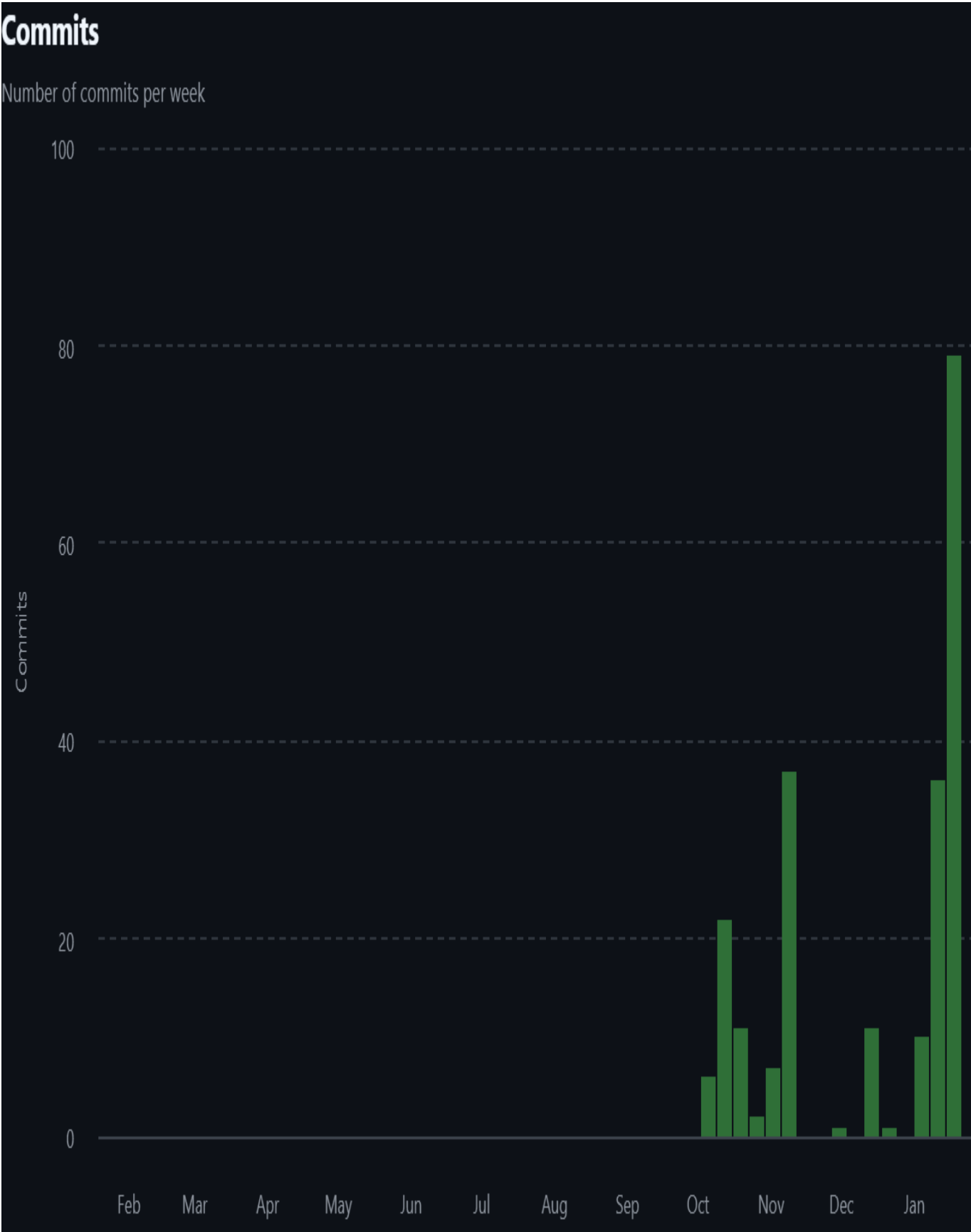
#	Aktivnost	Sati
1	Opis projektnog zadatka	4
2	Funkcionalni zahtjevi	3
3	Opis pojedinih obrazaca	3
4	Dijagram obrazaca	4
5	Sekvencijski dijagrami	2
6	Opis ostalih zahtjeva	3
7	Dijagram komponenti	1.5
8	Dijagram razmještaja	1
9	Zaključak i budući rad	1
10	Izrada prezentacije	0.5
11	Izrada wiki stranice	1

Kristijan Karanušić

#	Aktivnost	Sati
1	Funkcionalni zahtjevi	3
2	Arhitektura i dizajn sustava	4
3	Baza podataka	2
4	Dijagram razreda	2.5
5	Dijagram stanja	6
6	Korištene tehnologije i alati	1.5
7	Ispitivanje programskog rješenja	5
8	Dnevnik sastajanja	0.5
9	Popis literature	0.5

#	Aktivnost	Sati
10	Izrada prezentacije	3

Dijagram pregleda promjena



Programsko inženjerstvo ak.godina 2025./2026.

Sveučilište u Zagrebu

Fakultet elektrotehnike i računarstva

DanceArena

Tim: <14.1>

TheStickOfTruths

Nastavnik: Vlado Struk